

Companion to

Vibe Coding for
Medical Professionals

Exercise Help and Solutions

Part I — Hints

Part II — Solutions

Main-Book Exercises and
Python Companion Tasks

George Grätzer

Copy-editing manuscript edition
September 23, 2026

Edition: `medical-copyedit-2026-09-23`

About This Edition

This companion accompanies *Vibe Coding for Medical Professionals* and its *Python Companion*. Part I gives hints; Part II gives the worked solutions. The two parts follow the same chapter and task order, so a reader can consult a hint without seeing the solution beside it.

This edition covers all 98 numbered main-book exercises, all 23 chapter-level *Try One Change* tasks in the Python Companion, four larger practice tasks, and the integrated capstone. The numbering matches the main book. P1–P23 and R2, R8, R9, R23 are local navigation labels, explained in the introduction.

Edition identifier: `medical-copyedit-2026-09-23`. This identifier matches the main book and *Python Companion*. Both parts contain the same 98 numbered questions, including the large-data exercises and the clarified record-comparison question, Exercise 1.3.

The examples use synthetic administrative and classroom records with explicitly supplied teaching rules. They do not provide medical advice or establish suitability for operational use. No model sends messages, changes appointments, or authorizes real-data use. Open design questions have worked models, not uniquely correct implementations.

The final *Coverage and Checks* section distinguishes executed baseline checks from worked proposals and extension tasks. A large-file result does not replace a small fixture’s expected answer.

The LaTeX source embeds the complete manuscript. It needs only the supplied style file and ordinary LaTeX packages; neither other book, external chapter files, nor a figure folder is needed to typeset it. Runnable examples have their own stated dependencies.

Contents

How to Use This Companion	v
I Hints	1
1 Patient-Journey Timeline	3
2 Timeline Checker	5
3 From Eighteen to 12,000	7
4 Appointment Report	9
5 Measurement Catalogue	11
6 Measurement Checks	13
7 Weekly Teaching Extract	15
8 Follow-Up Matching	17
9 Cohort Summary	19
10 Monthly Waiting-Time Report	21
11 Weekly Follow-Up Monitor	23
12 Clinic Schedules	25
13 Appointment Letters	27
14 Rule-and-Boundary Lab	29
15 Predictive-Value Explorer	31
16 Literature-Review Organizer	33
17 Reproducible Research Table	35
18 Reminder Simulation	37
19 District Outreach	39
20 Journey Tests	41
21 Demonstration Package	43
22 Review-Status Check	45
23 Follow-Up Worklist	47
Larger Practice Files	49
The Integrated Capstone	51

II	Solutions	53
1	Patient-Journey Timeline	55
2	Timeline Checker	59
3	From Eighteen to 12,000	63
4	Appointment Report	67
5	Measurement Catalogue	71
6	Measurement Checks	75
7	Weekly Teaching Extract	79
8	Follow-Up Matching	83
9	Cohort Summary	87
10	Monthly Waiting-Time Report	91
11	Weekly Follow-Up Monitor	95
12	Clinic Schedules	99
13	Appointment Letters	103
14	Rule-and-Boundary Lab	107
15	Predictive-Value Explorer	111
16	Literature-Review Organizer	115
17	Reproducible Research Table	119
18	Reminder Simulation	123
19	District Outreach	127
20	Journey Tests	131
21	Demonstration Package	135
22	Review-Status Check	139
23	Follow-Up Worklist	143
	Larger Practice Files	147
	The Integrated Capstone	151
	Coverage and Checks	155

How to Use This Companion

Begin with the exercise in the main book or the Python Companion. Make a prediction and try the task before opening this volume. When you need a direction, turn to Part I. When you are ready to compare your work with a complete answer, turn to the matching entry in Part II.

Two Parts, One Numbering System

Both parts contain the same 23 chapter groups, followed by *Larger Practice Files* and *The Integrated Capstone*. In each numbered chapter, the main-book exercises come first and the Python Companion task follows. Thus Chapter 8 contains Exercises 8.1–8.5 and task P8 in both parts. Chapter numbering restarts at 1 in Part II to match the source books; the running heads say *Hints* or *Solutions*.

Main-book exercise numbers retain their original meaning. P1–P23 identify the unnumbered *Try One Change* sections in the corresponding Python Companion chapters. The practice labels R2, R8, R9, and R23 identify the four larger practice fixtures, not extra numbered main-book exercises. R9 includes the additional experiment that moves missing observations between groups. The capstone covers the original contract, a worked changed contract, and the Python Companion’s twenty-one-day example.

Consult a Hint Without Revealing the Answer

Part I contains each task followed by a short hint, without the worked answer or an answer-revealing title. Main-book questions reproduce the matching edition verbatim. Some Python Companion task statements already supply expected results; their Part I versions restate the assignment without those results. The full Python task wording is retained in Part II.

Each hint has a link and page reference to its solution. Each solution has a return link to its hint. For example, [Hint 7.3](#) is on page [15](#) and [Solution 7.3](#) is on page [80](#). Likewise, [Hint P12](#) and [Solution P12](#) refer only to the Python task, not to a main-book exercise. The PDF bookmarks provide direct access without expanding the printed contents into a long exercise list.

A hint suggests a record, distinction, or boundary to inspect. Stop there and try again. Read the solution afterwards to compare the reasoning, the result, and the checks. For a design question, a worked model is one defensible answer; another implementation may be acceptable if it meets the same stated contract and preserves the required evidence.

Run in a Fresh Copy

Create actual program and input files from the complete listings in the matching chapter of the *Python Companion*. Its run guide supplies the exact command and identifies any earlier modules required. Keep an unchanged copy, then create each changed trial in a fresh working folder. A refusal to overwrite an earlier output is often an expected protection.

The separate `journey_large.csv` supplies the large-data exercises. Do not replace a small fixture with it unless the exercise explicitly requires the change. Browser interaction and print-preview tasks need their own visual checks; file generation alone does not establish those results. The worked answers describe named reasoning and expected cases, not hidden model folders to download.

Part I

Hints

Patient-Journey Timeline

Exercise 1.1

Task. Reconcile P02's six records with C03 and C04. Name the event on each visit date.

Hint. *Separate the encounter references first; then read the date and type of each record.*

[Solution 1.1 \(page 55\)](#)

Exercise 1.2

Task. Add P04 with one visit on June 27 and two tests on June 28, all linked to new encounter C07 with distinct event IDs. Predict people, encounters, records, grouped markers, and the calendar range before running.

Hint. *A marker groups coincident events of the same type within a person, not every event on the same date.*

[Solution 1.2 \(page 55\)](#)

Exercise 1.3

Task. The input CSV has eighteen event records, but the browser count says nineteen. Compare the input with the exported CSV and the browser table by event ID, multiplicity, and source values. Explain how to distinguish a missing, extra, or duplicated record from an incorrect displayed count; do not assume which defect occurred.

Hint. *Compare both directions: records expected but absent, and records displayed but not supplied.*

[Solution 1.3 \(page 56\)](#)

Exercise 1.4

Task. Request an encounter filter as well as the person selector. Specify what the combination P01 and C03 should display, then test that it shows an explicit empty state rather than an earlier selection.

Hint. *Find C03's owner in the source and decide how two simultaneous selections should combine.*

[Solution 1.4](#) (page 56)

Python Companion P1

Task. In a fresh copy, change only E02's detail to "Sample collected: teaching revision." Run again. Check where the revised detail appears and decide which population totals should change, if any.

Hint. *Look for an unchanged population and a changed value in two representations.*

[Solution P1](#) (page 57)

Timeline Checker

Exercise 2.1

Task. Correct only the repeated E02 identifier. Explain why two rows change classification.

Hint. *Locate every occurrence of the repeated identifier. Ask whether uniqueness can be decided one row at a time.*

[Solution 2.1 \(page 59\)](#)

Exercise 2.2

Task. Add a second C06 visit with a distinct ID and an invalid date. Predict the status of both visits and their dependent events under the one-visit-row rule.

Hint. *The rule counts supplied visit rows before deciding whether either can anchor the encounter.*

[Solution 2.2 \(page 59\)](#)

Exercise 2.3

Task. A checker returns sixteen rows and says every returned row passes. Compare its identities with the input and explain why this can be a failure despite its claim.

Hint. *A statement about returned rows says nothing about records that were not returned.*

[Solution 2.3 \(page 60\)](#)

Exercise 2.4

Task. Construct one readable row with two independent issues. State the expected reasons and record count, then test it without changing the original file.

Hint. *Change a field on a row already known to have a different problem.*

[Solution 2.4 \(page 60\)](#)

Python Companion P2

Task. Change only the event ID in source row 14 from E02 to E14. Predict the placed, review, and total row counts, then check both source occurrences of the former duplicate.

Hint. *Recompute duplicate membership for both occurrences, not just the edited one.*

[Solution P2](#) (page 61)

From Eighteen to 12,000

Exercise 3.1

Task. Generate the P03 report and locate E18 with its preserved supplied person and separate display owner.

Hint. *The encounter visit determines display ownership; the original person field is not rewritten.*

[Solution 3.1 \(page 63\)](#)

Exercise 3.2

Task. Request a person reference that does not occur, such as P99. Predict the reference program's explicit failure and check that no output folder was created.

Hint. *Check validation and selection before creating the destination directory.*

[Solution 3.2 \(page 63\)](#)

Exercise 3.3

Task. A program filters to C01 before checking duplicates. Explain what happens to E02's duplicate evidence and write a record-level test that exposes the error.

Hint. *Look for matching identifiers outside the proposed encounter selection before choosing the order of operations.*

[Solution 3.3 \(page 64\)](#)

Exercise 3.4

Task. Add encounter selection after full-file validation. Save the choice in the run record and test the C01 result, an absent encounter, and an existing output folder.

Hint. *Extend the selection layer; leave whole-file validation and overwrite protection in place.*

[Solution 3.4 \(page 64\)](#)

Exercise 3.5

Task. A large P1000-only run reports twelve input records, twelve selected, twelve placed, and none for review. Which count is wrong? Name its three visit dates and explain why those checks are stronger than the selected count alone.

Hint. *Distinguish the input read by the checker from the records saved after selection. Compare P1000 with P0001, not merely with a total of twelve.*

[Solution 3.5 \(page 65\)](#)

Exercise 3.6

Task. In a copy of `journey_large.csv`, make only E000002's event date blank. Predict the complete and P0001-only classifications, identify what should change in the run records, and state how you will prove that the original input survived.

Hint. *Use the valid encounter visit to establish ownership. One date becomes missing; no row is deleted. Compare input and program identities separately.*

[Solution 3.6 \(page 65\)](#)

Python Companion P3

Task. In a fresh chapter copy, run `python3 journey_report.py journey_problem.csv P03_run --person P03`. Reconcile the selected P03 records, then repeat the identical command. Check whether any earlier output bytes changed. Use a different destination for another successful run.

Hint. *Check both the selected population and what happens to a repeated destination.*

[Solution P3 \(page 65\)](#)

Appointment Report

Exercise 4.1

Task. Reconstruct the three flag totals from the eight-row table and identify their overlap.

Hint. *Eligible, confirmed completed, and unresolved are flags, not three disjoint bins.*

[Solution 4.1](#) (page 67)

Exercise 4.2

Task. Add A08 with a valid scheduled date, status completed, and a missing completion date. Predict the new numerator, denominator, and unresolved count.

Hint. *Eligibility is decided before the completion-date check.*

[Solution 4.2](#) (page 67)

Exercise 4.3

Task. A report changes to 50 percent without a source change. Request its eligibility ledger and determine whether it dropped A03 or changed the cancellation policy.

Hint. *A 2/4 percentage does not identify which of the original five eligible rows disappeared.*

[Solution 4.3](#) (page 68)

Exercise 4.4

Task. Define a new report that excludes cancellations. Name the rows in its denominator and explain why its percentage can equal an incorrect report's percentage while using a different, explicitly stated rule.

Hint. *Write the new denominator before calculating its percentage.*

[Solution 4.4](#) (page 68)

Python Companion P4

Task. In the synthetic A03 row, supply completion date 2026-09-01. Predict the confirmed numerator, eligible denominator, and percentage. Check whether any unrelated ID or status classification changed.

Hint. *The source correction changes confirmation, not eligibility.*

[Solution P4](#) (page 69)

Measurement Catalogue

Exercise 5.1

Task. Trace the Celsius group to O02 and O04. State which source supplies the missing value.

Hint. *Look at measure, unit, and source separately.*

[Solution 5.1 \(page 71\)](#)

Exercise 5.2

Task. Add an observation of pulse for S03 in E04 with a new ID. Predict the observation, person, encounter, and group counts.

Hint. *Check whether the new observation's person and encounter references already occur in the source.*

[Solution 5.2 \(page 71\)](#)

Exercise 5.3

Task. A colleague adds distinct-person counts across the three groups and reports five people. Explain the repeated membership and give the correct overall count.

Hint. *Take the union of the person sets; do not add their sizes.*

[Solution 5.3 \(page 72\)](#)

Exercise 5.4

Task. Test an ID made only of spaces, an ID padded with spaces, and the ID 001. State which should be rejected and which source text must be preserved.

Hint. *Whitespace checking must not turn a source correction into an invisible normalization.*

[Solution 5.4 \(page 72\)](#)

Python Companion P5

Task. Copy the first observation as new synthetic observation O07. Change its value to 72, retaining the same person, encounter, measure, unit, and source. Predict the record, person, encounter, and group counts; then check the lineage file.

Hint. *The new ID adds a record while the existing references retain their meaning.*

[Solution P5](#) (page 73)

Measurement Checks

Exercise 6.1

Task. Account for all eight source records and the two separate M07 review rows.

Hint. *Distinguish a source row from an identifier value when counting repeated IDs.*

[Solution 6.1 \(page 75\)](#)

Exercise 6.2

Task. Give M04 both an invalid date and its missing value. Predict the number of rows, review rows, and reasons attached to M04.

Hint. *Inspect the reasons already attached to the row before adding the proposed change.*

[Solution 6.2 \(page 75\)](#)

Exercise 6.3

Task. A checker changes a finite value of 200 to 20 without a supplied rule. Explain what it has invented and what evidence or explicit teaching rule is needed instead.

Hint. *A finite number can be unusual without supplying evidence for a replacement.*

[Solution 6.3 \(page 76\)](#)

Exercise 6.4

Task. Add a documented teaching-range check. Specify two boundary cases, one outside case, and whether failure adds a review reason or rejects the file. Preserve the original values.

Hint. *Define units, endpoint inclusion, and the consequence of a failed check before testing.*

[Solution 6.4 \(page 76\)](#)

Python Companion P6

Task. For this invented fixture only, stipulate that M05's supplied unit label K was a transcription error and replace it with C. Predict and check the passing and review counts. Explain whether the change is a source correction or a unit conversion.

Hint. *Changing a unit label under a stipulation is different from converting a value.*

[Solution P6](#) (page 77)

Weekly Teaching Extract

Exercise 7.1

Task. Explain why E01 and E02 have an August week label although the extract selects September event dates.

Hint. *A Monday-based week can begin in the previous month.*

[Solution 7.1](#) (page 79)

Exercise 7.2

Task. Add a completed event on September 30 and another on October 1. Predict the included record, its week label, and the receipt counts.

Hint. *Both new events have the same Monday label, but only one is in September.*

[Solution 7.2](#) (page 79)

Exercise 7.3

Task. An extractor ignores an invalid August row and produces a September report. Explain why it violates this chapter's all-source validation contract, even when its counts look right.

Hint. *The contract validates the complete source before selecting September.*

[Solution 7.3](#) (page 80)

Exercise 7.4

Task. Design a rectangular week-by-status output with zeroes for absent categories. Specify the population of weeks and prove by reconciliation that the display change has not changed the included events.

Hint. *Choose the set of weeks explicitly and reconcile all cell counts to the same five events.*

[Solution 7.4](#) (page 80)

Python Companion P7

Task. Reverse the order of the six input records and run in another fresh copy. Compare the grouped-count CSV byte for byte with the baseline. Explain any expected difference between grouped output and a source-row ledger.

Hint. *Grouping is independent of the physical order of source rows.*

[Solution P7](#) (page 81)

Follow-Up Matching

Exercise 8.1

Task. For an index date of September 1, classify follow-ups on September 1, 15, and 16 under the inclusive fourteen-day rule.

Hint. *Subtract the index date; do not count the starting date as elapsed day one.*

[Solution 8.1 \(page 83\)](#)

Exercise 8.2

Task. Give E02 two eligible follow-ups on the same date with IDs F02 and F20. Predict the chosen ID and test a reversal of row order.

Hint. *The declared sort key is elapsed days followed by follow-up ID.*

[Solution 8.2 \(page 83\)](#)

Exercise 8.3

Task. Use one person with two encounters. Inspect a report that selects the same follow-up for both despite explicit different encounter references. Identify the wrong key and the lost distinction.

Hint. *A person may have several encounters with different explicitly linked follow-ups.*

[Solution 8.3 \(page 84\)](#)

Exercise 8.4

Task. Change earliest eligible to latest eligible. Write the new tie rule, one case where the chosen ID changes, and one where it does not. Explain why match counts alone are insufficient.

Hint. *Specify what happens to equal latest dates as well as unequal dates.*

[Solution 8.4 \(page 84\)](#)

Exercise 8.5

Task. Convert the large journey file using the stated field map. Reconcile every source event, compare fourteen-day and twenty-one-day matching, and use C000003 to explain why no match in a window does not mean no follow-up recorded.

Hint. *Separate the conversion ledger from the two matcher inputs. C000003 links an April 16 visit to a May 6 follow-up.*

[Solution 8.5 \(page 85\)](#)

Python Companion P8

Task. Run `python3 followup_match.py --window-days 15` in a fresh copy. Predict the new selections and matched count. Check E03, E04, and F07 individually, and state the changed teaching contract.

Hint. *The new upper endpoint admits day 15, but not negative elapsed days.*

[Solution P8 \(page 85\)](#)

Cohort Summary

Exercise 9.1

Task. Reconstruct the six observed waits and the median, and state the separate denominator for recorded follow-up.

Hint. *First identify eligible people; then distinguish follow-up availability from waiting-time availability.*

[Solution 9.1 \(page 87\)](#)

Exercise 9.2

Task. Remove only S10's waiting time. Predict eligibility and follow-up counts, then calculate the median from the five remaining observed waits.

Hint. *Change the value field only; leave age, enrolment, and follow-up status untouched.*

[Solution 9.2 \(page 87\)](#)

Exercise 9.3

Task. Two cohorts have identical observed medians but missing waits in different subgroups. Explain why equal observed medians do not establish equal overall experience.

Hint. *An observed median describes the supplied observations, not the values that are missing.*

[Solution 9.3 \(page 88\)](#)

Exercise 9.4

Task. Change the minimum age to 19. Write the new ordered exclusion rule, identify the newly excluded person, and require the ledger and summary to change together.

Hint. *S06 is exactly 18; retain the ordered reasons for every other candidate.*

[Solution 9.4 \(page 88\)](#)

Python Companion P9

Task. Stipulate an age of 18 for synthetic S07 and change only that field. Recalculate eligibility, recorded follow-up, observed waits, and the observed median. Check the contributing identities, not only the totals.

Hint. *A newly eligible person changes several populations at once.*

[Solution P9 \(page 89\)](#)

Monthly Waiting-Time Report

Exercise 10.1

Task. Reconcile January’s count and median with its two input records.

Hint. *Read the two January source rows rather than a plotted point.*

[Solution 10.1 \(page 91\)](#)

Exercise 10.2

Task. Add a January wait of 40 days. Predict its new mean and median and check that a comparison chart uses the same three records.

Hint. *Sort the three values for the median; add all three for the mean.*

[Solution 10.2 \(page 91\)](#)

Exercise 10.3

Task. A graph plots July at zero. Contrast that with a January containing two observed zero-day waits, and state the required point, gap, and count displays.

Hint. *The count distinguishes an absent measurement from a measured zero.*

[Solution 10.3 \(page 92\)](#)

Exercise 10.4

Task. Add an annotation for a fictional workflow change. Draft wording that records its date without claiming that the change caused the later pattern.

Hint. *An annotation can locate an event without explaining the subsequent pattern.*

[Solution 10.4 \(page 92\)](#)

Python Companion P10

Task. Add one invented July observation with wait 0. Predict July’s count and median and its representation on the chart. Compare the changed report with the baseline’s missing July.

Hint. *An added zero is an observation, not a way to fill an absent month automatically.*

[Solution P10 \(page 92\)](#)

Weekly Follow-Up Monitor

Exercise 11.1

Task. Reconstruct the 105/130 pooled result from the ten final weeks.

Hint. *Sum the ten final numerators and denominators separately.*

[Solution 11.1 \(page 95\)](#)

Exercise 11.2

Task. Use weeks with 1/2 and 90/100 completion. Calculate both the average weekly percentage and the pooled encounter percentage, and name the question each answers.

Hint. *An unweighted mean gives the two weeks equal weight despite their different sizes.*

[Solution 11.2 \(page 95\)](#)

Exercise 11.3

Task. All periods are provisional, but a report displays 0 percent final completion. Explain why the correct final rate is unavailable rather than observed zero.

Hint. *No final observations is not the same as final observations with no completions.*

[Solution 11.3 \(page 96\)](#)

Exercise 11.4

Task. Replace local week numbers with dated week starts in a reusable version. Define the year and week convention, reject duplicate periods, and keep the final-period selection unchanged.

Hint. *Use an unambiguous Monday date, not an unqualified number such as week 1.*

[Solution 11.4 \(page 96\)](#)

Python Companion P11

Task. Change only week 11's provisional flag to no. Recalculate the pooled final-period numerator, denominator, and percentage, and count the provisional weeks that remain. Leave every source count unchanged.

Hint. *Only the set of final periods changes.*

[Solution P11](#) (page 97)

Clinic Schedules

Exercise 12.1

Task. Trace the service clock for A, C, and E under urgent first, then reconcile their waits with the detail table.

Hint. *The current visit finishes before a newly arrived urgent visit can be selected.*

[Solution 12.1](#) (page 99)

Exercise 12.2

Task. Use only G arriving at 0 for five minutes and H arriving at 20 for five minutes, both routine. Predict both schedules, the idle interval, and the absent urgent-group mean.

Hint. *When nobody is ready, advance the clock to the next arrival.*

[Solution 12.2](#) (page 99)

Exercise 12.3

Task. A summary declares urgent first universally better because its overall mean is lower. Identify the subgroup and maximum-wait tradeoffs in this dataset.

Hint. *Compare subgroup means and the largest wait, not only the overall mean.*

[Solution 12.3](#) (page 100)

Exercise 12.4

Task. Add one administrative comparison measure of your choice, such as the number waiting over a specified time. State the cutoff and expected source identities before asking for code.

Hint. *Choose a strict or inclusive cutoff, then identify the contributing rows under each schedule.*

[Solution 12.4](#) (page 100)

Python Companion P12

Task. Set all six supplied priority labels to routine in a fresh copy. Compare the two schedules, their overall means, and their urgent-group summaries. Keep the arrival and service times unchanged.

Hint. *Remove the priority distinction without changing arrival or service times.*

[Solution P12](#) (page 101)

Appointment Letters

Exercise 13.1

Task. Match each of the three generated letters to its appointment ID and explain why L04 is held.

Hint. *The build report should name each generated or held request.*

[Solution 13.1](#) (page 103)

Exercise 13.2

Task. Give two appointments the same invented display name but different IDs and times. Predict the number of letters and check their separate identities.

Hint. *Use the appointment ID as the key, not the display name.*

[Solution 13.2](#) (page 103)

Exercise 13.3

Task. L02's letter uses the contact from L01. Specify a repair to the generator, not merely to the final page, and a check of every earlier letter.

Hint. *A field must be taken from the current request row each time the template is filled.*

[Solution 13.3](#) (page 104)

Exercise 13.4

Task. Add a review cover sheet and test a long invented name and location in print preview. Define what happens when a letter would otherwise spill onto an additional page.

Hint. *Decide what a long letter should do instead of shrinking or clipping it accidentally.*

[Solution 13.4](#) (page 104)

Python Companion P13

Task. Supply the invented contact 555-0101 for L04. Predict the generated and held request counts, then inspect the added letter's details. Do not add message-sending behavior.

Hint. *A supplied missing field changes generation eligibility, not the application's scope.*

[Solution P13](#) (page 105)

Rule-and-Boundary Lab

Exercise 14.1

Task. Reconstruct C03's three components rather than checking its total alone.

Hint. *Evaluate duration, task count, and support independently.*

[Solution 14.1](#) (page 107)

Exercise 14.2

Task. Keep duration at four and support at yes; test task counts 2, 3, 5, and 6. Predict all four totals.

Hint. *With duration four and support yes, only the task component varies.*

[Solution 14.2](#) (page 107)

Exercise 14.3

Task. A calculator gives C05 zero points. Explain why a missing duration is not a zero-duration observation and identify which output fields must be blank.

Hint. *The reference program calculates no component when any required input is invalid.*

[Solution 14.3](#) (page 108)

Exercise 14.4

Task. Change the upper duration cutoff to 12. Revise the expected cases for 10, 11.9, and 12 and explain why retaining every old expectation would be wrong.

Hint. *The middle duration interval now continues up to, but not including, 12.*

[Solution 14.4](#) (page 108)

Python Companion P14

Task. Change C02's duration from 5 to 4.9. Predict its duration component and total, then compare every case with the baseline. Use only the stated teaching intervals.

Hint. *The test crosses the lower boundary while every other case remains fixed.*

[Solution P14](#) (page 108)

Predictive-Value Explorer

Exercise 15.1

Task. At 10 percent prevalence, reconstruct all four expected counts and both condition-group totals.

Hint. *Use the condition-present and condition-absent populations separately.*

[Solution 15.1](#) (page 111)

Exercise 15.2

Task. Use 5 percent prevalence with the same model. Predict the true and false positive counts before comparing the browser and CSV calculation.

Hint. *Partition the supplied population into condition-present and condition-absent groups before applying the test assumptions.*

[Solution 15.2](#) (page 111)

Exercise 15.3

Task. A student says that 95 percent specificity makes 495 false positives impossible. Explain the mistake using the 9,900 people without the condition.

Hint. *Five percent of a large condition-absent population can still be many people.*

[Solution 15.3](#) (page 112)

Exercise 15.4

Task. Propose editable test characteristics. Define what the page must show when there are no expected positive results, and test that invalid input does not leave an old result on screen.

Hint. *Check the expected-positive denominator before displaying a predictive value.*

[Solution 15.4](#) (page 112)

Python Companion P15

Task. Add scenario rows with prevalence 0 and 100, keeping the baseline assumptions unchanged. Calculate all four expected cells, their total, and the positive predictive value for each row. Distinguish checks of the saved Python output from checks of the browser controls.

Hint. *The endpoints still have a nonzero expected-positive count under the fixed baseline characteristics.*

[Solution P15 \(page 113\)](#)

Literature-Review Organizer

Exercise 16.1

Task. Reconcile the eight retrieved records with the six study groups without deleting either repeated-study pair.

Hint. *Two studies each contribute two records; those extra records must remain traceable.*

[Solution 16.1 \(page 115\)](#)

Exercise 16.2

Task. Change only R02's year to 2025. Predict the decision counts and require the review output to name the field and both supplied values.

Hint. *A group can retain both records while reporting their disagreement.*

[Solution 16.2 \(page 115\)](#)

Exercise 16.3

Task. An organizer deletes a conference abstract because a later report has the same study ID. Explain what evidence can be lost and distinguish a study, report, and retrieved record.

Hint. *Shared study identity does not establish that two documents contain the same information.*

[Solution 16.3 \(page 116\)](#)

Exercise 16.4

Task. Design an extraction worksheet with an explicit row unit and stable source locators. State how pending and conflicting decisions affect entry into that worksheet.

Hint. *Choose the row unit before choosing the columns.*

[Solution 16.4 \(page 116\)](#)

Python Companion P16

Task. Change only R02's year from 2024 to 2023. Predict ST01's classification and the study-group totals. Identify the field involved in any disagreement while retaining both R01 and R02.

Hint. *Use the companion's year 2023, not the main exercise's year 2025.*

[Solution P16](#) (page 117)

Reproducible Research Table

Exercise 17.1

Task. Check S01's subtraction direction and reconstruct the five-change mean and median.

Hint. *The defined subtraction is follow-up minus baseline.*

[Solution 17.1](#) (page 119)

Exercise 17.2

Task. Remove S04's follow-up record instead of leaving its value blank. Predict which derived source locator changes and which complete-pair counts do not.

Hint. *The value is unavailable in both versions, but the source locator is different.*

[Solution 17.2](#) (page 119)

Exercise 17.3

Task. A report has mean change +2.8 but the same absolute magnitude as the reference. Name the smallest source-level check that detects the sign error.

Hint. *Use a pair whose two values are unequal.*

[Solution 17.3](#) (page 120)

Exercise 17.4

Task. Add a formatted summary with teaching units and one decimal place. Keep the unrounded CSV and specify how the formatted values will be checked against it.

Hint. *Keep the computational values separate from the strings used for display.*

[Solution 17.4](#) (page 120)

Python Companion P17

Task. Supply follow-up value 10 for S04's existing blank follow-up record. Predict the complete-pair count, S04's signed change, and the mean and median change. Keep the original input and run record as a separate version.

Hint. *A newly completed pair contributes another signed change. Rebuild the contributing list before calculating its summaries.*

[Solution P17 \(page 121\)](#)

Reminder Simulation

Exercise 18.1

Task. Recalculate the first two trial differences from the displayed attendance counts.

Hint. *The denominator is 80 in each group, and the result is in percentage points.*

[Solution 18.1 \(page 123\)](#)

Exercise 18.2

Task. Add a second plan row. State whether the declared single-plan program should reject it or run it, and test the actual response.

Hint. *The reference program explicitly requires exactly one data row in its plan file.*

[Solution 18.2 \(page 123\)](#)

Exercise 18.3

Task. A histogram places zero in the bin ending at zero even though bins include their left endpoint. Explain the boundary error and reconcile the corrected bin counts.

Hint. *A bin that includes its left endpoint excludes its right endpoint.*

[Solution 18.3 \(page 124\)](#)

Exercise 18.4

Task. Design a two-scenario comparison with scenario IDs, separate saved plans, and an explicit independence assumption. State which checks should remain unchanged when only the seed changes.

Hint. *Keep the scenario definition and the random experiment separate from the resulting summary.*

[Solution 18.4 \(page 124\)](#)

Python Companion P18

Task. Set both supplied probabilities to 0 and keep all other plan fields unchanged. Predict the attendance counts, trial differences, central bounds, and reminder-not-higher count before running the simulation.

Hint. *Check what attendance is possible at the probability boundary before considering the summary across trials.*

[Solution P18 \(page 125\)](#)

District Outreach

Exercise 19.1

Task. Reconstruct North's crude and equally weighted percentages from its two age-band counts.

Hint. *Calculate the two age-band rates before applying reference weights.*

[Solution 19.1](#) (page 127)

Exercise 19.2

Task. Use weights 0.7 and 0.3. Predict the four standardized results and identify which observed columns must not change.

Hint. *Use 0.7 for under 65 and 0.3 for 65-plus in every district.*

[Solution 19.2](#) (page 127)

Exercise 19.3

Task. North and South have equal standardized rates. Use their supplied age-specific rates to disprove the claim that the districts therefore have equal rates in each age band.

Hint. *Compare the age-specific rates directly.*

[Solution 19.3](#) (page 128)

Exercise 19.4

Task. Write a caption for a crude-versus-standardized chart that names the reference weights and explains why the standardized percentage is not an observed count.

Hint. *Name the weights and distinguish reweighting from an observed numerator.*

[Solution 19.4](#) (page 128)

Python Companion P19

Task. Double both reached and eligible counts in both North rows. Predict the changed district totals, crude percentage, and standardized percentage. Apply the multiplier to both counts in each row, not only to a numerator.

Hint. *Multiplying a numerator and its denominator by the same factor preserves their ratio.*

[Solution P19 \(page 128\)](#)

Journey Tests

Exercise 20.1

Task. Match each of the twelve test names with the behavior it is intended to establish.

Hint. *Read each assertion as a limited claim about a named fixture.*

[Solution 20.1](#) (page 131)

Exercise 20.2

Task. Disable the date-order check in an isolated copy and predict which named test must fail before running it.

Hint. *The problem fixture includes a follow-up before its visit.*

[Solution 20.2](#) (page 132)

Exercise 20.3

Task. A missing-visit test checks all returned C06 rows but not their identities or count. Explain how deleting the entire group can make this weak test pass.

Hint. *An all-rows assertion is true for an empty collection unless existence is checked separately.*

[Solution 20.3](#) (page 132)

Exercise 20.4

Task. Introduce that deletion defect in a separate copy. Add or inspect the completeness assertion, preserve the failed CSV row, restore the program, and rerun the suite.

Hint. *Delete the dependent rows only in an isolated faulty version; do not weaken the test.*

[Solution 20.4](#) (page 133)

Exercise 20.5

Task. In a copy of the large file, change E000014's event ID to E000002. Predict the all-person and P0001-only results. Explain how a filter-before-validation defect could escape a test that checks only twelve selected rows.

Hint. *The two occurrences now lie in different people's records. The source count and selected count can stay correct while the classifications become wrong.*

[Solution 20.5](#) (page 133)

Python Companion P20

Task. In a disposable copy of `journey_check.py`, change `days[i] < days[anchor]` to `days[i] <= days[anchor]`. Predict which named tests will detect the change, then run the suite in a fresh folder. Preserve the failure evidence, restore the original program, and rerun; do not alter expected answers to force a pass.

Hint. *Use a permitted same-day record to distinguish the two comparisons.*

[Solution P20](#) (page 133)

Demonstration Package

Exercise 21.1

Task. Explain why six source entries produce eight members in the baseline archive.

Hint. *Compare the source list with the archive manifest. Which members are created by the builder itself?*

[Solution 21.1](#) (page 135)

Exercise 21.2

Task. Add a release note and an actual test report to the source list. Predict both member counts and inspect the generated manifest.

Hint. *The test report must be generated by an actual test run, not a hand-written PASS label.*

[Solution 21.2](#) (page 135)

Exercise 21.3

Task. All hashes match, but RUN.txt names an obsolete program. Explain why file identity has not established a usable handover.

Hint. *A hash checks file identity, not the meaning of a sentence inside the file.*

[Solution 21.3](#) (page 136)

Exercise 21.4

Task. Package the worklist rather than the journey tool. Rewrite the file inventory and run instructions, then establish success by a fresh extraction and actual run.

Hint. *Identify the worklist's own program, input, dated command, and acceptance cases.*

[Solution 21.4](#) (page 136)

Exercise 21.5

Task. Starting from the eight-member baseline, add `journey_large.csv` and an independent `expected_large_summary.txt`, but no other files. Predict the source and archive counts, and explain which run instructions need revision before this is a usable large-data delivery.

Hint. *The two generated members remain separate from the supplied source list. Inspect what the old `RUN.txt` actually tells the recipient to execute.*

[Solution 21.5 \(page 137\)](#)

Python Companion P21

Task. Create an unrelated `decoy.txt` beside the program without adding it to `package_files.csv`. Build the ZIP in a fresh copy. Predict its member count and whether it contains the decoy, then inspect the archive.

Hint. *A file beside the builder is not automatically a file in the package.*

[Solution P21 \(page 137\)](#)

Review-Status Check

Exercise 22.1

Task. Reconcile three blocker records with four reasons in the original register.

Hint. *Count rows and reason occurrences separately, inspecting rows that carry more than one reason.*

[Solution 22.1](#) (page 139)

Exercise 22.2

Task. Blank both current and reviewed versions in Specification. Predict its classification under the nonblank-version rule.

Hint. *The rule requires nonblank versions as well as equality.*

[Solution 22.2](#) (page 139)

Exercise 22.3

Task. An accepted review expired before the reporting date. Explain why the decision string alone does not make the register row complete.

Hint. *The decision string is one condition among several.*

[Solution 22.3](#) (page 140)

Exercise 22.4

Task. Prepare an action list for your synthetic project that retains the reporting date, classifier rule version, artifact versions, and unresolved questions without inventing reviewer decisions.

Hint. *Translate blockers into questions and required evidence, not fictitious approvals.*

[Solution 22.4](#) (page 140)

Python Companion P22

Task. In the invented Program row, change `reviewed_version` from 3 to 4 so that it matches `current_version`. Recalculate complete and blocker counts and inspect the reasons on the other rows. Explain what this stipulated register change does and does not establish.

Hint. *This is a stipulated register correction, not evidence that a real review occurred.*

[Solution P22 \(page 141\)](#)

Follow-Up Worklist

Exercise 23.1

Task. Identify the four baseline status groups by encounter ID, not only by their counts.

Hint. *A review reason takes precedence over ordinary window-status classification.*

[Solution 23.1 \(page 143\)](#)

Exercise 23.2

Task. Give E01 a follow-up on September 17, then September 18 in a separate copy. Explain the valid late record and the future-record review case.

Hint. *The worklist records valid known follow-up even after the matching window.*

[Solution 23.2 \(page 143\)](#)

Exercise 23.3

Task. The capstone matcher and worklist each contain eleven positive cases. Use C103 and C114 to show why those counts do not establish that the reports use the same definition.

Hint. *One encounter enters the recorded set without entering the matched set; another does the reverse.*

[Solution 23.3 \(page 144\)](#)

Exercise 23.4

Task. Complete stage B from your clarified brief. Deliver the specification, field map, row ledger, dated outputs, tests, one failed case and repair, and a package that succeeds after fresh extraction.

Hint. *Resolve the two policy questions before adapting the program or the tests.*

[Solution 23.4 \(page 144\)](#)

Exercise 23.5

Task. Design a worklist conversion from the large journey file as of September 17, 2026. What happens when contact route and owner are left blank? What changes when office and Teaching Team are explicitly supplied as fictional administration? Contrast both results with fourteen-day matching without inventing weekly status flags.

Hint. *Review reasons take precedence. The source supplies recorded dates but not the administrative fields. A late recorded follow-up is not the same as no recorded follow-up.*

[Solution 23.5](#) (page 145)

Python Companion P23

Task. Run `python3 followup_worklist.py --as-of 2026-09-25` in a fresh copy. Predict every status group and check E03 in particular. Compare E03's classification on September 24 with its classification on September 25.

Hint. *At the window-end date the inclusive window is still open.*

[Solution P23](#) (page 145)

Larger Practice Files

Use the larger input files in the *Larger Practice Files* section of the *Python Companion*. Their programs are the chapter programs, but their records differ from the small teaching examples. Predict the result before opening the supplied expected-result files. The R labels identify the associated chapters.

R2: Thirty-Six Journey Records

Task. Reconcile every record in the Chapter 2 fixture in *Larger Practice Files* in the *Python Companion*. Explain the duplicate identifier and the difference between the supplied person and display owner.

Hint. *Use source-row locators: the repeated identifier cannot identify both occurrences uniquely.*

[Solution R2 \(page 147\)](#)

R8: Matching Forty Source Records

Task. Match sixteen encounters with 24 follow-up records. Account for early and late dates, equal-day ties, repeat people, and the unknown encounter reference.

Hint. *Group by encounter, apply the inclusive zero-through-fourteen-day window, then choose by elapsed day and follow-up ID.*

[Solution R8 \(page 148\)](#)

R9: Move the Missing Observations

Task. Reconcile the forty-person cohort. Then move the same pattern of missing waits to another subgroup without changing the observed multiset. Explain why equal observed medians do not establish equal experience for the whole cohort.

Hint. *Keep eligibility, recorded follow-up, and observed wait as separate properties of a person. Swap supplied waits rather than inventing values for blanks.*

[Solution R9 \(page 148\)](#)

R23: Thirty-Six Worklist Encounters

Task. Reconcile the larger worklist as of September 17, 2026, with a fourteen-day inclusive window. Check exact status identities and all review reasons.

Hint. *Count review records separately from their reasons. Compare a valid late follow-up with an absent follow-up.*

[Solution R23](#) (page 149)

The Integrated Capstone

Task. Connect the matcher, weekly monitor, and worklist using the supplied capstone inputs and an explicit field map. Complete main-book Exercise 23.4 and the integrated application in the Python Companion. Keep the original reporting contract separate from each changed contract, and deliver the specification, source ledger, dated outputs, tests, failure-and-repair evidence, and a package that runs after fresh extraction.

[The complete worked capstone \(page 151\)](#)

Stage A: Start with the Fixed Contract

The supplied reporting date is September 17, 2026. Matching includes elapsed days zero through fourteen. All encounters enter the denominator for the Monday week containing their index date. Retain the supplied final/provisional flags.

Hint. *Draw separate sets for selected within-window matches, valid recorded follow-ups, and review cases. Ask whether the sets must coincide or be disjoint. Keep the source-row ledger separate from these encounter-level sets. Reconcile identities before comparing totals.*

[The Stage-A solution \(page 151\)](#)

Stage B: State the Changed Question

Write the changed reporting decisions in a new project card before altering the adapter or its expected results. To compare with this companion's worked model, retain the fourteen-day matching window and use a closed-window weekly denominator.

Hint. *Write the denominator rule as a date inequality. Decide how it treats an encounter on its window-end date, and which numerator and review records belong to that denominator. A record outside a denominator still belongs in the complete source ledger.*

[The Stage-B solution \(page 152\)](#)

The Python Companion's Twenty-One-Day Change

In another fresh copy, use a twenty-one-day window and a closed-window denominator, keeping the reporting date fixed. The Python Companion supplies the adapter command. Work out the new expectations before consulting its outputs.

Hint. *Calculate the earliest window-end date in the source and compare it with the reporting date. Keep three questions separate: whether a follow-up matches, whether it has been validly recorded, and whether its encounter enters the weekly denominator.*

[The twenty-one-day solution](#) (page 153)

Failure, Repair, and Handover

Preserve one deliberate failure and its repair, then test a freshly extracted copy of the package using its printed run instructions.

Hint. *Choose a case where recorded follow-up and within-window matching differ. A test that checks only an aggregate count may miss the defect. Restore the program rather than editing expected answers to manufacture a pass, and keep the original and changed-contract tests distinguishable.*

[The failure-and-repair solution](#) (page 153)

[The handover discussion](#) (page 154)

Part II

Solutions

Patient-Journey Timeline

Solution 1.1: Two Encounters, Six Records

[Hint 1.1 \(page 3\)](#)

Task. Reconcile P02’s six records with C03 and C04. Name the event on each visit date.

Solution. C03 contains E07, the visit on June 3; E08, the test on June 4; and E09, the follow-up on June 10. C04 contains E10, the visit on June 15; E11, the measurement on June 15; and E12, the follow-up on June 22. All six records belong to P02.

The visit dates are therefore June 3 and June 15. On June 15 the input supplies two events: the visit and a measurement. They are separate records even though their dates agree. The file gives no time of day, so their placement on different event-type lanes does not establish which occurred first.

Check the selected page against this six-ID list, not merely against the displayed total of six. Match the CSV and the page’s detail table by event ID, encounter, date, and type. A duplicated record and an omitted record could leave the total unchanged.

Solution 1.2: A Fourth Person

[Hint 1.2 \(page 3\)](#)

Task. Add P04 with one visit on June 27 and two tests on June 28, all linked to new encounter C07 with distinct event IDs. Predict people, encounters, records, grouped markers, and the calendar range before running.

Solution. One suitable addition uses E19 for P04’s visit on 2026-06-27 and E20 and E21 for the two tests on 2026-06-28. All three carry encounter C07 and nonblank invented details. These IDs are new in this fixture.

The predicted totals are four people, seven encounters, and 21 records. The original page has 18 markers: its same-day visit and measurement records have different types and therefore different markers. P04 adds two markers, one for the visit and one grouping the two tests. The full page thus has 20 grouped markers representing 21 records. The grouped test marker must expose both E20 and E21 in its details.

The global calendar range changes from June 1–26 to June 1–28. Selecting P04 should show three records on two markers, on the page’s shared calendar scale. The two tests must also remain separate rows in the exported CSV and detail table.

Solution 1.3: A Count Is Not an Identity Check

Hint 1.3 (page 3)

Task. The input CSV has eighteen event records, but the browser count says nineteen. Compare the input with the exported CSV and the browser table by event ID, multiplicity, and source values. Explain how to distinguish a missing, extra, or duplicated record from an incorrect displayed count; do not assume which defect occurred.

Solution. The discrepancy alone does not identify a defect. An incorrect counter can report nineteen even when the table and export each retain the eighteen input records. Alternatively, a table or export can contain missing, extra, or duplicated records. Compare the actual records before deciding which situation applies.

Construct a record-level comparison of the eighteen source IDs with the page's displayed event records and with the exported CSV. Compare multiplicities as well as membership: a set alone hides a repeated ID. List missing IDs, extra IDs, and IDs occurring the wrong number of times. For matched IDs compare the supplied person, encounter, date, type, and detail.

Also check the counter's definition. It must count records in the current selection, not markers, rows accumulated across earlier selections, or hidden sections. This comparison identifies missing, extra, or duplicated records, if present, and separately checks the displayed count. A correct repair must fix the generator or selector and regenerate the output.

Solution 1.4: Combined Filters and an Empty Result

Hint 1.4 (page 4)

Task. Request an encounter filter as well as the person selector. Specify what the combination P01 and C03 should display, then test that it shows an explicit empty state rather than an earlier selection.

Solution. Specify that a row is selected only when it matches every active filter. A blank filter means all values for that field. For P01 together with C03 the selected set is empty, because every C03 row belongs to P02. The page must show zero records and an explicit message such as "No records match this person and encounter." It must not retain a previous timeline or detail table.

Use this acceptance sequence: select P01 alone and obtain E01–E06; add C01 and obtain E01–E03; change only the encounter to C03 and obtain no records; change the person to P02 and obtain E07–E09; clear both filters and recover all eighteen. Repeat in the opposite selection order.

The implementation should derive the selected rows from the original validated collection on each change and rebuild the count, markers, and detail rows from that

same collection. Clear old output before rendering an empty selection. Keeping this as a new version avoids changing the Chapter 1 reference application.

Solution P1: Try One Change

[Hint P1 \(page 4\)](#)

Task. In a fresh copy, change only E02’s detail to “Sample collected: teaching revision.” Run again. The new text must appear in both the CSV and browser page, while the totals remain three people, six encounters, and 18 events. This separates a presentation change from a change in the population being displayed.

Solution. Edit only E02’s detail in a fresh input copy to “Sample collected: teaching revision.” Run `journey_view.py`. E02 must carry the exact new text in `journey_events.csv`, and the generated HTML must contain it in the event details and marker information. The three-person, six-encounter, eighteen-record totals remain unchanged; there are still eighteen markers in this fixture.

Compare all other source fields, particularly E02’s date and references, with the original. A new record or changed identifier would be a different experiment. The automated check executes the program and examines the generated CSV and HTML text. In the browser, also select P01 and inspect E02’s displayed details; that interactive check is distinct from finding the text in the file.

Timeline Checker

Solution 2.1: One Correction, Two Reclassifications

[Hint 2.1 \(page 5\)](#)

Task. Correct only the repeated E02 identifier. Explain why two rows change classification.

Solution. Change the event ID of source row 14 from E02 to E14, leaving row 2 as E02. Both IDs then occur once. Row 2 and row 14 each lose the reason *duplicate event_id*; both become placed because neither has another problem in this fixture.

There are still eighteen input and output records. The placed count increases from twelve to fourteen and the review count falls from six to four. Review rows 3, 8, 11, and 18 remain, for the before-visit date, missing date, impossible date, and person mismatch respectively.

Check both rows explicitly. An implementation that updates only the edited row could leave row 2 incorrectly flagged and report thirteen placed records. Recompute the duplicate-ID counts from the entire changed input, then classify every row again. Preserve the original problem file and its six-review-row output as a separate baseline.

Solution 2.2: An Invalid Second Visit Still Makes Two Visits

[Hint 2.2 \(page 5\)](#)

Task. Add a second C06 visit with a distinct ID and an invalid date. Predict the status of both visits and their dependent events under the one-visit-row rule.

Solution. For example, append E19, person P03, encounter C06, type visit, date 2026-06-31, and a nonblank detail. The impossible date does not make this supplied visit row disappear. C06 now has two visit rows, E16 and E19, so neither is an unambiguous anchor.

E16 receives *multiple visits for encounter*. E19 receives that reason and *invalid event_date*. E17 and E18 receive *no valid visit for encounter*. All four C06 records are retained for review. There are nineteen records overall, ten placed and nine for review.

E18 illustrates a further consequence. Without a valid C06 anchor, the checker cannot establish P03 as the display owner or diagnose a disagreement with that anchor. Its supplied P01 is preserved and used as the fallback owner. The earlier

person-mismatch reason is replaced by the missing-valid-anchor reason; it must not be retained as though the old anchor were still valid.

Solution 2.3: Sixteen Passing Rows Can Be a Failed Report

[Hint 2.3 \(page 5\)](#)

Task. A checker returns sixteen rows and says every returned row passes. Compare its identities with the input and explain why this can be a failure despite its claim.

Solution. The input contains eighteen records. A sixteen-row report has already failed the conservation requirement, even when all sixteen returned rows genuinely pass their checks. The missing records may be exactly the difficult ones that the checker was supposed to preserve for review.

Use source filename and data-row number as locators. The problem fixture repeats E02, so event ID alone is not a unique locator. Require one output row for each input data row 1–18, with no extra or repeated source-row locator. Compare every original field at each locator before inspecting the added status and reasons.

Then compare classifications with the expected six review positions: 2, 3, 8, 11, 14, and 18. The checker must explain troublesome records rather than discard them. A report may separately provide a placed-only display, but its complete ledger and advertised total must still account for all eighteen source records.

Solution 2.4: Two Reasons on One Record

[Hint 2.4 \(page 5\)](#)

Task. Construct one readable row with two independent issues. State the expected reasons and record count, then test it without changing the original file.

Solution. In a fresh copy of the problem fixture, change source row 3, E03, from person P01 to P02. Leave its date as 2026-05-31 and its encounter as C01. C01's valid visit E01 belongs to P01 on June 1. E03 therefore has two independent reasons: *person differs from visit* and *event before visit*.

The full report still contains eighteen rows, twelve placed and six for review: E03 was already a review row before this change. What increases is the number of reason occurrences, from six to seven, not the number of review rows. The reference checker lists the person mismatch before the before-visit reason on this record.

Verify that both reasons remain attached to E03's one source-row locator and that all source fields survive unchanged. A checker using an exclusive choice between these two tests would wrongly suppress one reason. Conversely, duplicating E03 into two output records would violate the one-record-per-source-row rule.

Solution P2: Try One Change

[Hint P2 \(page 6\)](#)

Task. Change only the event ID in source row 14 from E02 to E14. Both duplicate-ID reasons should clear, leaving 14 placed rows and four review rows. The row count stays 18. A result of 13 placed rows would suggest that only one occurrence was reconsidered.

Solution. This is the same controlled source correction as main-book Exercise 2.1: replace only source row 14's E02 by E14. Run `journey_check.py` in the fresh copy. The checked CSV must contain eighteen rows, fourteen placed and four for review. Neither former occurrence may retain a duplicate-ID reason.

Locate the remaining review records by source positions 3, 8, 11, and 18. Do not let sorting the output persuade you that row positions are event IDs. The automated check verifies the eighteen-record total, fourteen placed rows, and removal of the duplicate reason from the complete report.

From Eighteen to 12,000

Solution 3.1: Source Person and Display Owner

[Hint 3.1 \(page 7\)](#)

Task. Generate the P03 report and locate E18 with its preserved supplied person and separate display owner.

Solution. Run `python3 journey_report.py journey_problem.csv P03_run --person P03` in a fresh chapter copy. The program validates all eighteen input records, then selects the six whose derived owner is P03. Four are placed and two need review.

The selected records are source rows 13–18: E13, the repeated E02, E15, E16, E17, and E18. In E18’s saved row, `person_ref` must remain P01 while `owner_ref` is P03. The review reason is *person differs from visit*. C06’s valid visit E16 establishes P03 as the owner used for this report.

Inspect `P03_run/journey_report.csv`, the review section of its HTML, and `run_record.json`. The run record must distinguish eighteen input records from six selected records. Filtering on the supplied person before checking would misplace E18 and lose the evidence this exercise is meant to retain.

Solution 3.2: An Absent Person Must Not Create a Report

[Hint 3.2 \(page 7\)](#)

Task. Request a person reference that does not occur, such as P99. Predict the reference program’s explicit failure and check that no output folder was created.

Solution. Use a new destination name and request `--person P99`. P99 does not occur among the report owners. The reference program raises a `ValueError` rather than creating a normal report for an invented person. Its command exits unsuccessfully, and the requested output directory must not exist.

Record the actual error and inspect the parent folder. An empty directory left behind would not satisfy this exercise’s stated expectation, even though no report files were written. Do not confuse this command-line failure policy with the explicit empty browser state requested in Exercise 1.4; these are different interface contracts.

A regression test should begin with a destination known not to exist, perform the request, require the exception or nonzero exit, and assert that the path still does not

exist. Run the valid P03 request afterward with another fresh destination to show that the invalid request did not damage the source files.

Solution 3.3: Validate Before Selecting

[Hint 3.3 \(page 7\)](#)

Task. A program filters to C01 before checking duplicates. Explain what happens to E02's duplicate evidence and write a record-level test that exposes the error.

Solution. C01 contains source rows 1–3, with E02 at row 2. Its duplicate is at row 14 in C05. Selecting C01 before counting identifiers hides that second occurrence, so a defective checker would place C01's E02. The correct full-file validation marks it for review even in a C01-only report.

Test the final selected identities, not only the counts. The correct C01 result has E01 placed; source row 2, E02, for review because of the duplicate identifier; and E03 for review because it precedes the visit. Thus it has three records, one placed and two for review.

Also compare the full-file classification of source row 2 with its classification in the selected report: status and reasons must agree. The selection may reduce what is displayed, but it must not change which source records were considered when evaluating a whole-file constraint.

Solution 3.4: A Reusable Encounter Selection

[Hint 3.4 \(page 7\)](#)

Task. Add encounter selection after full-file validation. Save the choice in the run record and test the C01 result, an absent encounter, and an existing output folder.

Solution. A suitable revised command adds `--encounter C01`. Read the complete input and call the existing checker first. Only afterward keep rows satisfying every supplied owner/person and encounter filter. Save the selected encounter, the person choice if any, input total, selected total, status totals, and the input/program identities in the run record.

For C01 alone require the three source rows and classifications established in Exercise 3.3: E01 placed, E02 and E03 for review. For a nonexistent encounter such as C99, use the same explicit-failure-before-directory-creation policy as the reference program's absent-person request. Define a combined selection that has no rows in the same way, rather than leaving its behavior accidental.

For an existing destination, require failure and compare every pre-existing file byte for byte before and after. Finally, generate a successful C01 report in another new folder. These tests protect three separate requirements: complete validation, correct selection, and preservation of earlier output. Changing the filter must not silently change any of them.

Solution 3.5: Large-Data Check

Hint 3.5 (page 8)

Task. A large P1000-only run reports twelve input records, twelve selected, twelve placed, and none for review. Which count is wrong? Name its three visit dates and explain why those checks are stronger than the selected count alone.

Solution. The input count must be 12,000. The selected, placed, and review counts are 12, 12, and 0. P1000 has visits on March 11, May 18, and July 8, 2026, in C002998, C002999, and C003000. Their follow-ups are March 26, June 8, and July 24. A program could copy another person's twelve rows and pass the count check; these identities and dates expose that mistake. The full input must be checked before selecting P1000.

Solution 3.6: Large-Data Check

Hint 3.6 (page 8)

Task. In a copy of `journey_large.csv`, make only E000002's event date blank. Predict the complete and P0001-only classifications, identify what should change in the run records, and state how you will prove that the original input survived.

Solution. The complete run retains 12,000 records: 11,999 placed and one for review. P0001 selects twelve records: eleven placed and E000002 for review with the missing-date reason. Both runs still examine 12,000 input records. The changed copy has a different input hash, while the three unchanged program hashes agree with the baseline. Recompute the original source hash and compare its bytes or hash with the preserved download; do not infer preservation merely because the copy has a different name. Use new output folders for both trials.

Solution P3: Try One Change

Hint P3 (page 8)

Task. Run `python3 journey_report.py journey_problem.csv P03_run --person P03` in a fresh chapter copy. Check the P03 counts above. Then repeat the identical command: it must fail rather than overwrite `P03_run`. Verify that the earlier files are unchanged. For another successful run, choose a different destination name.

Solution. The first P03 command succeeds with eighteen input records, six selected records, four placed, and two for review. Its six source locators and E18's two distinct person fields are described in Exercise 3.1.

Before repeating the identical command, record the bytes or hashes of every file in `P03_run`. The repeat must fail with the existing-directory condition and leave those bytes unchanged. Merely seeing an error is insufficient: a defective program could overwrite one file and only then fail on another.

The automated check performs the successful run, inspects E18 and the selected counts, repeats the command, and compares the earlier files. A third run may succeed only under a different destination name. Do not delete the first result just to make the second command succeed.

Appointment Report

Solution 4.1: Overlapping Report Flags

[Hint 4.1 \(page 9\)](#)

Task. Reconstruct the three flag totals from the eight-row table and identify their overlap.

Solution. Five rows are eligible: A01, A02, A03, A04, and A05. Two rows have confirmed completion with a valid date: A01 and A02. Four source rows are unresolved: A03, A06, and both occurrences of A07. Thus the three totals are 5 eligible, 2 completed, and 4 unresolved, against eight source rows.

A03 belongs to both the eligible population and the unresolved group. Its recognized completed status and valid appointment identity/date establish eligibility, but its missing completion date prevents confirmation. A06 has an unrecognized status. Both A07 rows are excluded from eligibility because their appointment ID is duplicated.

The completed rows are already among the eligible rows, so adding the three totals is meaningless. Reconcile by source row and flag combination instead. The baseline proportion is 2/5, or 40 percent; the unresolved flag does not authorize silently shrinking that denominator.

Solution 4.2: An Eligible Appointment with Unresolved Completion

[Hint 4.2 \(page 9\)](#)

Task. Add A08 with a valid scheduled date, status completed, and a missing completion date. Predict the new numerator, denominator, and unresolved count.

Solution. Append A08 with a new unpadded ID, a valid scheduled date such as 2026-09-01, status completed, and a blank completion date. It is eligible under the unchanged rule, but its completion is not confirmed. It also receives the missing-or-invalid-completion-date reason.

The result has nine source rows, numerator 2, denominator 6, and five unresolved source rows. The proportion is 2/6, approximately 33.3 percent. The new appointment is the additional member of both the denominator and the unresolved set; no previous row changes classification.

Check A08's three flags explicitly: eligible 1, completed 0, unresolved 1. A report retaining a denominator of five has probably dropped the missing outcome, while a

numerator of three has probably mistaken a supplied status word for a confirmed dated completion. Both are errors under this report's definition.

Solution 4.3: The Same Percentage from Different Rows

[Hint 4.3 \(page 9\)](#)

Task. A report changes to 50 percent without a source change. Request its eligibility ledger and determine whether it dropped A03 or changed the cancellation policy.

Solution. Request the eligibility ledger, the exact numerator IDs, the denominator IDs, and the stated rule version. Under the unchanged source, dropping unresolved A03 leaves A01, A02, A04, and A05 and produces 2/4, or 50 percent. That contradicts the original policy of retaining eligible rows with unconfirmed outcomes.

Excluding cancellations instead removes A04 and retains A01, A02, A03, and A05. It also produces 2/4, but answers a newly defined question. It is not a correction of the original report unless the change of definition has been explicitly requested and recorded.

Compare the two identity lists with the baseline A01–A05. The percentage alone cannot distinguish the mistakes or decisions. With no source change and no authorized rule change, the original result remains 2/5. Require the program to expose the cause of the changed percentage rather than accept 50 percent because it appears plausible.

Solution 4.4: An Explicit Cancellation Policy

[Hint 4.4 \(page 9\)](#)

Task. Define a new report that excludes cancellations. Name the rows in its denominator and explain why its percentage can equal an incorrect report's percentage while using a different, explicitly stated rule.

Solution. Define the new population as appointments satisfying the original identity, scheduled-date, and recognized-status rules, except that cancelled appointments do not enter the denominator. Keep the original rule for a confirmed completed appointment. Preserve all eight source rows in the ledger, including the newly excluded cancellation.

The denominator is A01, A02, A03, and A05. The numerator remains A01 and A02, so the new report is 2/4, or 50 percent. A03 stays eligible and unresolved; A04 is excluded specifically by the new cancellation policy. A06 and the two A07 rows retain their original problems.

This numerical agreement with the incorrect A03-dropping report in Exercise 4.3 is accidental. The reports use different four-row populations. Label the revised rule and display numerator and denominator identities or provide an accessible ledger. A change note should say what question changed, not claim that a higher percentage means the data improved.

Solution P4: Try One Change

[Hint P4 \(page 10\)](#)

Task. In the synthetic A03 row, supply completion date 2026-09-01. The confirmed numerator becomes 3 while the eligible denominator remains 5: 60.0%. Seven or eight eligible rows would indicate that unrelated ID or status rules changed as well.

Solution. Supply 2026-09-01 as A03's completion date in the invented fixture. A03 was already one of the five eligible appointments; it now also becomes a confirmed completion. The new result is 3/5, or 60.0 percent, with three unresolved source rows: A06 and the two A07 occurrences.

Run the unchanged `appointment_report.py` and inspect A03's flags, which should be eligible 1, completed 1, unresolved 0. The input still contains eight source rows. This is a stipulated correction to synthetic data, not information inferred from the fact that the status said completed. Record that distinction in the change note.

Measurement Catalogue

Solution 5.1: Tracing the Celsius Group

[Hint 5.1 \(page 11\)](#)

Task. Trace the Celsius group to O02 and O04. State which source supplies the missing value.

Solution. The Celsius temperature group consists of O02 and O04. O02 is S01's observation in E01, value 37.0, from `teaching_A`. O04 is S02's observation in E03, with no supplied value, from `teaching_B`. The missing value therefore comes from O04 in `teaching_B`.

The catalogue row must show two observations, one supplied value, and one missing value, with both sources retained. The lineage file must contain both observation IDs and their person, encounter, group, and source references. The Fahrenheit observation O06 belongs to a different measure–unit group and must not be used to fill O04.

A grouped count is an inventory statement, not a conversion or imputation. The correct result locates the missing value without inventing a temperature for it.

Solution 5.2: More Observations without More People

[Hint 5.2 \(page 11\)](#)

Task. Add an observation of pulse for S03 in E04 with a new ID. Predict the observation, person, encounter, and group counts.

Solution. Add O07 for pulse, person S03, encounter E04, unit beats/min, a supplied invented value such as 70, and an explicit source such as `teaching_A`. The overall counts become seven observations, three people, four encounters, and three measure–unit groups.

The pulse group grows from three observations to four and now includes S03 as well as S01 and S02. Its supplied-value count grows to four when the new value is nonblank. The Celsius and Fahrenheit groups are unchanged. The lineage file gains exactly one row identifying O07 and its references.

Do not report a new person merely because this is S03's first pulse observation. Conversely, do not suppress O07 because S03 and E04 already occur: those references are allowed to repeat. The uniqueness rule applies to observation IDs.

Solution 5.3: Group Membership Is Not a Partition of People

[Hint 5.3 \(page 11\)](#)

Task. A colleague adds distinct-person counts across the three groups and reports five people. Explain the repeated membership and give the correct overall count.

Solution. The pulse group contains S01 and S02. The Celsius temperature group also contains S01 and S02. The Fahrenheit temperature group contains S03. Their distinct-person counts are therefore 2, 2, and 1, which add to five group memberships but not five distinct people.

The union is {S01, S02, S03}, so there are three people overall. S01 and S02 each appear in two groups. This is legitimate repeated membership, not a duplicate-person error.

A useful revised display labels each per-group count as “distinct people in this group” and calculates the overall distinct-person count directly from the complete source or lineage collection. It must not calculate that overall count by summing group rows. The same caution applies whenever the same person may contribute observations of several types.

Solution 5.4: Identifier Text Must Survive

[Hint 5.4 \(page 11\)](#)

Task. Test an ID made only of spaces, an ID padded with spaces, and the ID 001. State which should be rejected and which source text must be preserved.

Solution. An observation ID containing only spaces is rejected because it is blank after whitespace checking. A padded ID such as “ O07 ” is also rejected under the declared nonblank, unpadded-ID contract. Do not silently strip it and continue; that would change the supplied identifier without recording a correction.

The ID 001 is valid when it is new and unique. Preserve it as the three-character text 001; do not convert it to the integer 1 or write it back as 1. These identifiers label records rather than represent quantities.

Make three independent input copies and change the same chosen observation ID in each. The first two runs should fail before catalogue files are created. The third should succeed with unchanged record, person, encounter, and group totals, and with 001 present verbatim in the lineage. Preserve the original rejected source text in the input copy and the error record.

Solution P5: Try One Change

[Hint P5 \(page 12\)](#)

Task. Copy the first observation as a new synthetic observation O07, changing its value to 72 but keeping the same person, encounter, measure, unit, and source. Expect seven observations, still three people and four encounters, still three groups, and four observations in the pulse group. The lineage file must gain O07.

Solution. Copy O01 as O07 and set its value to 72, leaving person S01, encounter E01, measure pulse, unit beats/min, and source teaching__A unchanged. The original O01 already has value 72, so the new ID is the actual new piece of information in this fixture; the exercise does not create a new numerical value.

The output must have seven observations, three people, four encounters, and three groups. The pulse catalogue row has four observations and four supplied values. The lineage file contains both O01 and O07. It is not the program's job to collapse two different observation IDs merely because their other fields agree. The automated check verifies the new lineage length, group totals, and person/encounter counts.

Measurement Checks

Solution 6.1: Eight Source Records, Not Seven Identifiers

[Hint 6.1 \(page 13\)](#)

Task. Account for all eight source records and the two separate M07 review rows.

Solution. M01 and M02 pass the declared checks. M03 has an impossible date, February 30; M04 has a missing value; M05 has the unrecognized supplied unit K; and M06 has the nonnumeric value warm. Source rows 7 and 8 both carry M07 and both receive the duplicate-ID reason.

The result therefore contains eight rows: two pass and six require review. There are only seven distinct supplied IDs, but that does not authorize returning seven records. The two M07 rows contain different dates and values and need their separate data-row locators.

Reconcile each output row with its original row, including both M07 values, 37.2 and 37.3. The checker may add status and reasons; it must neither keep only the first repeated ID nor average the repeated rows. Such changes would destroy the evidence the exercise asks you to inspect.

Solution 6.2: More Reasons, Not More Records

[Hint 6.2 \(page 13\)](#)

Task. Give M04 both an invalid date and its missing value. Predict the number of rows, review rows, and reasons attached to M04.

Solution. Set M04's event date to an impossible date such as 2026-02-30 while leaving its value blank. M04 then has two reasons: invalid event date and missing value. Both belong to source row 4.

The report still has eight records, with two passing and six for review. It has seven reason occurrences overall instead of six. A count of reasons and a count of affected records answer different questions; neither should be labelled simply “errors” without explanation.

Inspect M04 directly and ensure both issues are present. Then check that no second output row was created for it and that the other seven source rows are unchanged. This distinguishes cumulative validation from an implementation that stops after the first problem or multiplies output rows to represent several problems.

Solution 6.3: An Unsupported Repair

Hint 6.3 (page 13)

Task. A checker changes a finite value of 200 to 20 without a supplied rule. Explain what it has invented and what evidence or explicit teaching rule is needed instead.

Solution. The supplied checks do not justify changing 200 to 20. Both are finite numerical values. The checker has invented a decimal-place correction without source evidence or a declared teaching rule. A suspicious result is not proof of the intended result.

Preserve 200 and identify its source locator. A real source correction would need appropriate source verification and an authorized, recorded change; this teaching exercise can instead stipulate an invented correction and label it explicitly. A range rule, when supplied, can flag the value for review but still does not determine the replacement.

Keep original value, proposed or authorized revised value, reason, source/version, and the basis for the change in a separate record. Do not describe an automatic division by ten as data checking when no such transformation was part of the contract. The goal is to expose uncertainty, not make the file look tidy.

Solution 6.4: A Declared Teaching Range

Hint 6.4 (page 13)

Task. Add a documented teaching-range check. Specify two boundary cases, one outside case, and whether failure adds a review reason or rejects the file. Preserve the original values.

Solution. One permissible model is a purely invented teaching rule: a supplied Celsius value must lie from 35 through 40 inclusive. The rule is not a clinical reference range. It applies only after the value parses as finite and the supplied unit is C. Keep the existing date, ID, missing-value, and unit checks.

Use new distinct records with valid dates and values 35, 40, and 40.1. The first two pass the added range test; 40.1 acquires *outside supplied teaching range*. A Fahrenheit row is not compared with Celsius cutoffs, and a missing or nonnumeric value does not become zero for this test.

In this model, failure adds a review reason rather than rejecting the entire readable file. Preserve the exact supplied value and unit in every output row. Test an outside-range record with a separate date problem as well: both reasons should appear. Save the range definition and rule version with the result so the new flag can be interpreted and reproduced.

Solution P6: Try One Change

Hint P6 (page 14)

Task. For this invented fixture only, stipulate that M05's supplied unit label K was a transcription error and replace it with C. Expect three rows passing the checks and five review rows. This is a stated source correction, not an automatic conversion between units; the program performs no such conversion.

Solution. The exercise explicitly stipulates that synthetic M05's K was a transcription error. Change only that label to C and run `measurement_check.py`. Its numerical text remains 38.0; do not subtract 273.15 or otherwise transform the value.

M05 now passes, so the eight-row report has three passing records and five review records. The remaining review rows are M03, M04, M06, and the two M07 occurrences. The change note must say that the exercise author supplied the correction. Without that stipulation, automatically replacing K with C would not be justified.

Weekly Teaching Extract

Solution 7.1: Month Selection and Week Labels

[Hint 7.1 \(page 15\)](#)

Task. Explain why E01 and E02 have an August week label although the extract selects September event dates.

Solution. E01 on September 1 and E02 on September 2 both fall in the week whose Monday is August 31, 2026. They are included because selection uses each event's own September date. The grouped output's week label describes the Monday of the containing week, not the event's month.

E05 is different: its event date is August 31, so it is outside the September population even though it has the same week-start label. Thus the week beginning August 31 contains two included September events, not three source events.

The five included events are E01, E02, E03, E04, and E06. Their grouping creates one completed and one no-show event in the first week, and two completed plus one cancelled event in the week starting September 7. This reconciles the selection rule with a week label that initially appears surprising.

Solution 7.2: The Last Day of the Month

[Hint 7.2 \(page 15\)](#)

Task. Add a completed event on September 30 and another on October 1. Predict the included record, its week label, and the receipt counts.

Solution. Add a completed event E07 dated 2026-09-30 and another E08 dated 2026-10-01, using distinct IDs and the required invented fields. September 30 is included; October 1 is excluded. Both dates belong to the week starting Monday, September 28.

The receipt should now report eight input records, six included September records, and two outside the period: the original E05 and the new E08. The grouped output gains a September 28 completed row with count one. Existing grouped counts stay unchanged.

Verify the receipt against the source identities, not only the new group total. Including both new events because their week starts in September would answer a

different selection question. The month test must precede or remain logically separate from the grouping by Monday.

Solution 7.3: Outside the Period Is Not Outside Validation

[Hint 7.3 \(page 15\)](#)

Task. An extractor ignores an invalid August row and produces a September report. Explain why it violates this chapter’s all-source validation contract, even when its counts look right.

Solution. The chapter requires all-source validation. An invalid August row must therefore cause the source-validation failure even though it would later be outside the September extract. Ignoring it violates the declared contract; correct-looking September totals do not excuse that change in behavior.

A concrete test changes E05’s date in a fresh copy to 2026-08-32. The extractor should fail on the impossible date rather than generate the usual September output. Use a new working folder so a previously generated CSV cannot be mistaken for the result of this failed run.

A different contract could validate only a carefully defined subset, but it would need to say how records with unreadable selection fields are handled. It cannot be introduced silently as an optimization. Keep the distinction between a deliberate outside-period exclusion and a source record that failed validation.

Solution 7.4: A Rectangular Display without a Population Change

[Hint 7.4 \(page 15\)](#)

Task. Design a rectangular week-by-status output with zeroes for absent categories. Specify the population of weeks and prove by reconciliation that the display change has not changed the included events.

Solution. A complete calendar model uses every Monday-based week intersecting September 2026: August 31 and September 7, 14, 21, and 28. Use the three recognized statuses completed, no_show, and cancelled. This gives fifteen cells, including explicit zeros. It is a display extension, not permission to include August or October events.

For August 31 the counts are completed 1, no_show 1, cancelled 0. For September 7 they are completed 2, no_show 0, cancelled 1. Every cell in the remaining three weeks is zero in the baseline fixture. Summing all fifteen cells gives five, equal to the unchanged included-event count; status totals are completed 3, no_show 1, cancelled 1.

Construct the rectangle from the defined weeks and statuses, then look up observed counts with a zero default. Preserve the receipt’s six input, five included, and one outside-period counts. This explicitly chosen calendar population differs from the also defensible choice of only observed weeks; the heading must say which is used.

Solution P7: Try One Change

[Hint P7 \(page 16\)](#)

Task. Reverse the order of the six input records and run in another fresh copy. The grouped-count CSV should be unchanged byte for byte. Input order is not part of this report's grouping rule, although a future source-row ledger would need to record the new row positions.

Solution. Reverse all six input rows and run `minimal_extract.py` in a fresh copy. The reference program sorts its grouped keys before writing `weekly_counts.csv`, so that output should be identical byte for byte to the baseline grouped CSV, not merely have the same total.

The receipt remains six input records, five selected September records, and one outside-period record. E05 remains excluded. This task concerns a grouped report with no source-row locators. In an application that did save such locators, reversing the input would legitimately change those positions; one would compare stable identities and classifications instead of demanding unchanged row-number fields. The automated check makes the byte comparison for this specific grouped output.

Follow-Up Matching

Solution 8.1: Inclusive Calendar Boundaries

[Hint 8.1 \(page 17\)](#)

Task. For an index date of September 1, classify follow-ups on September 1, 15, and 16 under the inclusive fourteen-day rule.

Solution. For an index date of September 1, a follow-up on September 1 is day 0 and is eligible. September 15 is day 14 and is also eligible. September 16 is day 15 and is outside the inclusive fourteen-day window.

The condition is that elapsed calendar days lie between 0 and 14, with both endpoints included. An event before September 1 is not eligible. The phrase “fourteen-day rule” should not be allowed to hide either endpoint convention; write both explicitly in the specification and tests.

Check these as named candidate records with different encounter references or in separate input copies so that the earliest-choice rule does not obscure eligibility. A row can be eligible without being selected when an earlier eligible row exists for the same encounter.

Solution 8.2: A Tie that Does Not Depend on Row Order

[Hint 8.2 \(page 17\)](#)

Task. Give E02 two eligible follow-ups on the same date with IDs F02 and F20. Predict the chosen ID and test a reversal of row order.

Solution. Give E02 two eligible candidates F02 and F20 on the same date, for example September 15. Their elapsed days tie at 14. The program selects F02 because its ID sorts before F20. F20 remains in the candidate ledger as eligible but not selected by the day/ID rule.

Reverse the input row order and rerun in another fresh folder. E02 must still select F02. Its candidate count should be two in both runs. The candidate CSV may preserve a different physical order, but the chosen ID and each candidate’s eligibility and reason must agree after matching by follow-up ID.

Test the actual string-ID convention rather than an unstated numeric interpretation of the suffix. These particular IDs give the same result either way, but the specification

should remain clear for future inputs. Choosing the first row encountered is not an implementation of the declared tie rule.

Solution 8.3: The Encounter Is the Matching Key

[Hint 8.3 \(page 17\)](#)

Task. Use one person with two encounters. Inspect a report that selects the same follow-up for both despite explicit different encounter references. Identify the wrong key and the lost distinction.

Solution. Use person S01 with encounters E01 and E02. Give F01 an explicit E01 reference and F02 an explicit E02 reference, both on eligible dates. A correct report selects each follow-up only for its stated encounter. It does not pool both candidates under S01 and reuse the earlier one twice.

The defective key is the person reference. It collapses the distinct encounter relationships and can falsely describe a later encounter as followed up using an event belonging to an earlier one. The appropriate candidate filter is equality of the supplied encounter reference with the encounter being evaluated.

A direct test requires E01's selected ID to be F01 and E02's to be F02, with the candidate ledger showing the same links. The two matched totals alone cannot detect the error: both the correct and defective reports might claim two matches.

Solution 8.4: Latest Eligible Is a New Selection Rule

[Hint 8.4 \(page 17\)](#)

Task. Change earliest eligible to latest eligible. Write the new tie rule, one case where the chosen ID changes, and one where it does not. Explain why match counts alone are insufficient.

Solution. One precise new rule is: choose the greatest eligible elapsed day; among candidates on that day, choose the lexicographically smallest follow-up ID. Implement it by ordering on decreasing elapsed day and increasing ID, not by simply taking the last item of the old increasing-day/increasing-ID sort. That shortcut would reverse the ID tie rule too.

E05 changes from F05 on September 4 to F06 on September 6. E01 remains F01 because it has only one eligible candidate. E02 remains F02 in the baseline. The number of matched encounters remains three, so totals alone cannot show whether the revised rule was implemented.

Save the new selection rule in the specification and preserve each candidate's eligibility. Compare selected IDs, elapsed days, and reasons by encounter. Add a same-latest-date pair to test the declared ID tie rule independently of the earliest-to-latest change.

Solution 8.5: Large-Data Check

[Hint 8.5 \(page 18\)](#)

Task. Convert the large journey file using the stated field map. Reconcile every source event, compare fourteen-day and twenty-one-day matching, and use C000003 to explain why no match in a window does not mean no follow-up recorded.

Solution. The conversion ledger has 12,000 rows, each preserving one source event. Its dispositions are 3,000 visits used as encounters, 3,000 follow-ups used as candidates, and 6,000 tests or measurements not used for matching. At fourteen days there are 1,684 matches and 1,316 encounters without an in-window match; at twenty-one days there are 3,000 matches and none without one. Both trials have zero unknown encounter references. C000003's E000009 visit is April 16 and its E000012 follow-up is May 6, twenty days later. The recorded follow-up survives in both trials; only the window decision changes. Save each window choice and both reports without changing the source events.

Solution P8: Try One Change

[Hint P8 \(page 18\)](#)

Task. Run `python3 followup_match.py --window-days 15` in a fresh copy. E03 should now select F03 and the matched encounter count should rise from three to four. E04 remains before the index date; F07 remains an unknown reference. The changed result belongs to a different, explicitly stated teaching contract.

Solution. Run `python3 followup_match.py --window-days 15`. E03's F03 on September 16 becomes eligible and selected. The matched encounters are now E01, E02, E03, and E05; E04 and E06 remain unmatched. Thus the count rises from three to four of six encounters.

F04 remains before E04's index date and F07 remains attached to unknown E99. E05 still chooses F05 under the unchanged earliest-day/ID selection rule. Inspect those unchanged cases as well as the new match. This is a changed teaching contract, not a correction to the fourteen-day output; both versions should remain available with their window lengths stated.

Cohort Summary

Solution 9.1: Three Different Denominators

[Hint 9.1 \(page 19\)](#)

Task. Reconstruct the six observed waits and the median, and state the separate denominator for recorded follow-up.

Solution. The eight eligible people are S01, S02, S04, S06, S08, S09, S10, and S12. The six observed waits are S01: 4, S02: 8, S06: 10, S08: 12, S10: 6, and S12: 14 days. Sorted, these are 4, 6, 8, 10, 12, 14. The median is the average of the two middle values, 8 and 10, giving 9 days.

The waiting-time median uses six observed values, not eight eligible people or twelve candidates. Recorded follow-up is a separate proportion: six of the eight eligible people have follow-up recorded, or 75 percent. S04 and S09 remain eligible despite their missing waits and absent recorded follow-up.

The same six people happen to supply both kinds of information in this small baseline, but the definitions are not interchangeable. Preserve the eligibility ledger and name the basis of each statistic in the summary.

Solution 9.2: Removing One Wait without Removing a Person

[Hint 9.2 \(page 19\)](#)

Task. Remove only S10's waiting time. Predict eligibility and follow-up counts, then calculate the median from the five remaining observed waits.

Solution. Blank S10's waiting time in a fresh copy. S10 remains eligible and still has recorded follow-up. There are therefore eight eligible people and six recorded follow-ups, just as before. The observed-wait count decreases from six to five.

The remaining waits, in increasing order, are 4, 8, 10, 12, and 14. Their middle value is 10, so the new observed median is 10 days. The follow-up proportion remains $6/8$, or 75 percent. The missing-wait count among eligible people becomes three.

Check S10's included disposition in the ledger and the separate basis counts in the summary. A change to seven eligible people would mean the program had silently made observed waiting time an eligibility condition. Retaining a median of 9 would mean the old value had not actually been removed from the calculation.

Solution 9.3: Equal Observed Medians, Different Missingness

Hint 9.3 (page 19)

Task. Two cohorts have identical observed medians but missing waits in different subgroups. Explain why equal observed medians do not establish equal overall experience.

Solution. Two cohorts can share an observed median even when the available observations represent different parts of their populations. For example, each might have ten eligible people and the same eight observed waits, while the two missing values belong to one subgroup in the first cohort and another subgroup in the second. The matching medians say nothing about those four unobserved experiences.

Report eligible counts, observed counts, missing counts, and the distribution of missingness across any supplied relevant subgroups. Do not replace a missing wait with zero or assert that its distribution matches the observed waits without an additional justified assumption.

The defensible conclusion is that the two *observed* waiting-time medians agree under the stated inclusion rules. Equal overall experience is a stronger claim and is not established by these data. An answer should name this limit rather than attempt to calculate an unsupported population median.

Solution 9.4: Changing the Age Threshold

Hint 9.4 (page 19)

Task. Change the minimum age to 19. Write the new ordered exclusion rule, identify the newly excluded person, and require the ledger and summary to change together.

Solution. A revised ordered rule is: first assign unresolved age when age is missing; next exclude a supplied age below 19; then exclude those not enrolled; otherwise include the person. Preserve the input-validation rules and record this threshold as a new cohort definition.

S06, age 18, is newly excluded. S03, age 17, remains below the threshold; S07 still has unresolved age; S05 and S11 remain not enrolled. The new flow is one unresolved age, two below 19, two not enrolled, and seven included, summing to twelve candidates.

The seven included people are S01, S02, S04, S08, S09, S10, and S12. Five have recorded follow-up. Their five observed waits are 4, 6, 8, 12, and 14, so the median is 8 days. Change the ledger, threshold wording, summary, and expected counts together. This is a supplied teaching policy, not an age-based clinical recommendation.

Solution P9: Try One Change

[Hint P9 \(page 20\)](#)

Task. Stipulate an age of 18 for synthetic S07 and change that one field. Expect nine eligible people, seven recorded follow-ups, and seven observed waits. Their median becomes 8 days. This change affects both cohort membership and the observed-value list, so retaining the old median would be a warning sign.

Solution. Stipulate age 18 for S07 in the synthetic source. Under the original age-at-least-18 rule, S07 changes from unresolved age to included. The report now has nine eligible people, seven recorded follow-ups, and seven observed waits.

Those waits sort to 4, 5, 6, 8, 10, 12, 14, with median 8. The follow-up fraction is $7/9$, approximately 77.8 percent. This is not the same change as Exercise 9.4, which raises the minimum age to 19; keep the two experiments in different folders.

The automated check inspects the changed summary values. Also inspect S07 in the disposition ledger and keep the original blank-age input, making clear that the exercise supplies a new fact rather than the program inferring a person's age.

Monthly Waiting-Time Report

Solution 10.1: January from Its Two Observations

[Hint 10.1 \(page 21\)](#)

Task. Reconcile January's count and median with its two input records.

Solution. January contains waits of 4 and 8 days. Its count is two and its median is their midpoint, 6 days. The mean also happens to be 6, but that equality is a feature of these two observations, not the definition of a median.

The January row of `monthly_waits.csv` must therefore show count 2 and median 6.0. Its plotted point should be at six days. The two source rows, the summary row, and the chart must all refer to the same month and population.

A count of one would suggest a lost record or an accidental collapse by month before collecting the values. A point at four or eight would suggest that one observation was selected instead of calculating the median. This small hand calculation is enough to expose those defects.

Solution 10.2: A Large Value Changes the Mean More

[Hint 10.2 \(page 21\)](#)

Task. Add a January wait of 40 days. Predict its new mean and median and check that a comparison chart uses the same three records.

Solution. Append a third January observation of 40 days. The values are 4, 8, and 40. Their sum is 52, so the mean is $52/3$, approximately 17.33 days. The median is the middle sorted value, 8 days. The January count becomes three.

The reference monthly program reports the median, so its changed January row should show count 3 and median 8. A mean comparison is an additional output that must be specified; it should not silently replace the median series. Label both measures and derive both from the same three source records.

A comparison chart should put the two January values at 8 and about 17.33, with units of days. Retain the unrounded mean in the computational result or enough precision to reproduce it, and round only for the chosen display. Other months should remain unchanged.

Solution 10.3: No Observation and an Observed Zero

[Hint 10.3 \(page 21\)](#)

Task. A graph plots July at zero. Contrast that with a January containing two observed zero-day waits, and state the required point, gap, and count displays.

Solution. The baseline has no July observations. Its monthly row has count zero and an unavailable median, represented by a blank field rather than the numerical value zero. The chart should show a gap for July, with no July point and no line segment falsely bridging the missing month as an observed result.

In the contrasting January fixture, two supplied zero-day waits give count two and median zero. That month should have a real plotted point at zero, connected according to the ordinary chart rule for adjacent observed months.

A useful check examines count, value, and display together: July is (0, unavailable, gap), while the changed January is (2, 0, point). A zero-height point is not an adequate representation of missing data. Nor should an actual zero observation be discarded merely because a conditional test treats zero as false.

Solution 10.4: An Annotation without a Causal Claim

[Hint 10.4 \(page 21\)](#)

Task. Add an annotation for a fictional workflow change. Draft wording that records its date without claiming that the change caused the later pattern.

Solution. One suitable label is: “Fictional workflow change introduced on 1 August 2026. The line marks its date; this descriptive series does not establish its effect on waiting times.” Use a vertical line or clearly positioned note at the declared date, while leaving the underlying observations and monthly summaries unchanged.

Keep the distinction between event timing and evidence of causation. This exercise has not supplied a comparison group, an analysis of other changes, or a causal design. An annotation such as “New process reduced waits” would assert more than the chart supports.

Check the note against the chosen date and make sure it neither hides the missing July interval nor shifts the August point. The annotation should also survive print preview without covering the axis labels or the numerical series.

Solution P10: Try One Change

[Hint P10 \(page 21\)](#)

Task. Add one invented July observation with wait 0. July should now have count 1 and median 0, and the chart should have an actual July point. This is deliberately different from the baseline’s missing July. The exercise tests representation of supplied zero, not a proposal to fill missing real records with zero.

Solution. Append exactly one invented July row with `wait_days` 0. The unchanged monthly program should now report July count 1 and median 0. The observed-month count increases from eleven to twelve, and the missing-month list becomes empty.

The chart must now contain a genuine July point at zero. This differs from the original July gap and from Exercise 10.3's two observed January zeros. Do not alter any other month or replace every blank month by a zero.

The automated check verifies July's changed count and median. Inspect the SVG as well to confirm the intended point and adjacent segments. The stipulation that a zero-day observation was supplied belongs in the source-change note; it is not an imputation rule.

Weekly Follow-Up Monitor

Solution 11.1: A Pooled Final-Period Result

[Hint 11.1 \(page 23\)](#)

Task. Reconstruct the 105/130 pooled result from the ten final weeks.

Solution. The final completed counts are 8, 9, 7, 11, 12, 10, 9, 13, 12, and 14, which sum to 105. Their eligible counts are 10, 12, 10, 14, 15, 12, 11, 16, 14, and 16, which sum to 130. The pooled result is therefore 105/130, approximately 80.769 percent, displayed as 80.8 percent.

Weeks 11 and 12 are provisional and do not enter either sum. They remain visible in the weekly report. Unresolved counts are retained separately; they do not silently reduce the supplied eligible denominators.

Reconcile the ten included week identities as well as the two totals. Averaging the ten displayed percentages, especially after rounding them, would calculate a different quantity. Pooling gives each eligible encounter the same weight under the supplied weekly counts.

Solution 11.2: Average Week and Pooled Encounter

[Hint 11.2 \(page 23\)](#)

Task. Use weeks with 1/2 and 90/100 completion. Calculate both the average weekly percentage and the pooled encounter percentage, and name the question each answers.

Solution. The two weekly percentages are $1/2 = 50$ percent and $90/100 = 90$ percent. Their unweighted average is 70 percent. That answers the question: what is the mean of these two weekly percentages when each week has equal weight?

Pooling instead gives 91 completed among 102 eligible encounters, or about 89.216 percent. That answers the question: what proportion of all eligible encounters in the two weeks were completed? Displayed to one decimal place, the pooled value is 89.2 percent.

Neither calculation is an arithmetic mistake when its question is stated correctly. They differ because the 100-encounter week contributes fifty times as many encounters as the two-encounter week. For this book's final monitor the declared answer is the pooled encounter proportion, so the program must sum the counts before dividing.

Solution 11.3: No Final Denominator

[Hint 11.3 \(page 23\)](#)

Task. All periods are provisional, but a report displays 0 percent final completion. Explain why the correct final rate is unavailable rather than observed zero.

Solution. When every period is provisional, the selected final population is empty. The final completed total and final eligible total are both zero, but their ratio is undefined. The correct final-rate display is unavailable or “No final-period denominator,” not an observed zero percent.

Retain all provisional periods and report their count. A reader should be able to distinguish this case from a final week with zero completions among ten eligible encounters, which genuinely has a zero-percent rate.

Test a fresh copy with every provisional flag set to yes. Require a blank or explicitly unavailable pooled value and an explanatory label. The implementation must branch on the denominator before division; it must not use a zero fallback that erases the difference between no selected observations and an observed absence of completions.

Solution 11.4: Dated Weeks in a Reusable Monitor

[Hint 11.4 \(page 23\)](#)

Task. Replace local week numbers with dated week starts in a reusable version. Define the year and week convention, reject duplicate periods, and keep the final-period selection unchanged.

Solution. One precise model replaces the local week column with `week_start`, an exact YYYY-MM-DD date that must be a Monday. A period covers that Monday through the following Sunday. The date carries its year, including for weeks crossing a calendar-year boundary; no separate week-number convention is needed.

Reject duplicate week-start dates and non-Monday dates before writing reports. Sort valid periods by date. Retain the supplied completed, eligible, unresolved, and provisional fields and their existing validity rules. The final selection remains exactly those rows whose provisional flag is no.

For the original twelve rows, assign twelve consecutive Monday dates in a declared year and verify that the same ten final records give 105/130. Also test a duplicated Monday, a Tuesday, and two different years with otherwise similar labels. The capstone’s week-map file illustrates an alternative adapter approach when the original numbered-week program is kept unchanged.

Solution P11: Try One Change

[Hint P11 \(page 24\)](#)

Task. Change only week 11's provisional flag to no. The final totals become 113 completions among 142 eligible records, or 79.6%; one provisional week remains. Do not change the counts merely to keep the original percentage.

Solution. Change week 11's provisional flag from yes to no, leaving its counts at eight completed among twelve eligible and two unresolved. The final totals become 113/142, approximately 79.577 percent, displayed as 79.6 percent. There are now eleven final weeks and one provisional week.

The final percentage decreases from 80.8 even though the completed total increases. This is expected: the added week's completion fraction is lower than the previous pooled fraction. The revised result must come from adding both its numerator and its denominator.

The automated check verifies the new counts, percentage, and remaining provisional count. Keep week 12's flag and every numerical source field unchanged.

Clinic Schedules

Solution 12.1: Tracing the Service Clock

[Hint 12.1 \(page 25\)](#)

Task. Trace the service clock for A, C, and E under urgent first, then reconcile their waits with the detail table.

Solution. Under the nonpreemptive urgent-first rule, A starts at minute 0 and finishes at 20; its wait is zero. By minute 20 both urgent visits C and E are available. C arrived earlier, so it starts at 20 and finishes at 25, waiting 10 minutes. E then starts at 25 and finishes at 30, waiting 5 minutes.

The complete urgent-first order is A, C, E, B, D, F. B runs from 30 to 40 and waits 25; D runs from 40 to 50 and waits 28; F runs from 50 to 60 and waits 25. The six waits sum to 93, giving mean 15.5 minutes.

C does not interrupt A at minute 10. This distinguishes the supplied rule from a preemptive system, which would be a different simulation. Check each detail row's arrival, start, finish, and subtraction before trusting the summary.

Solution 12.2: An Idle Server and an Absent Subgroup

[Hint 12.2 \(page 25\)](#)

Task. Use only G arriving at 0 for five minutes and H arriving at 20 for five minutes, both routine. Predict both schedules, the idle interval, and the absent urgent-group mean.

Solution. With only G arriving at 0 for five minutes and H arriving at 20 for five minutes, both rules produce the same schedule. G starts at 0, finishes at 5, and waits zero. The server is idle from minute 5 to minute 20. H starts at 20, finishes at 25, and also waits zero.

The overall mean wait, maximum wait, and routine-group mean are all zero. The urgent-group mean is blank or unavailable because there are no urgent visits. It is not an observed urgent mean of zero.

Check that the program does not start H at minute 5 merely because service is available then: H has not yet arrived. Conversely, it must not treat the fifteen idle minutes as H's waiting time. The exercise tests both advancement of the clock and the handling of an empty subgroup.

Solution 12.3: Lower Overall Mean Is Not Universal Improvement

[Hint 12.3 \(page 25\)](#)

Task. A summary declares urgent first universally better because its overall mean is lower. Identify the subgroup and maximum-wait tradeoffs in this dataset.

Solution. First-come service has overall mean wait 18 minutes, urgent-group mean 22.5, routine-group mean 15.75, and maximum wait 25. Urgent-first service has overall mean 15.5, urgent-group mean 7.5, routine-group mean 19.5, and maximum wait 28.

Thus the urgent-labelled group waits less and the overall mean falls, while the routine-labelled group's mean rises and the largest wait increases. In particular, B's wait rises from 15 to 25 and D's from 23 to 28. These are the tradeoffs hidden by the single overall average.

The output describes this invented arrival file under two supplied administrative queue rules. It does not establish that one scheduling policy is universally better or justify using its labels for clinical prioritization. A balanced summary names the measures and the affected groups without turning the exercise into a policy recommendation.

Solution 12.4: A Measure with Named Contributing Visits

[Hint 12.4 \(page 25\)](#)

Task. Add one administrative comparison measure of your choice, such as the number waiting over a specified time. State the cutoff and expected source identities before asking for code.

Solution. One suitable additional measure is the number of visits waiting *strictly more than 20 minutes*. Under first-come service these are D, E, and F, with waits 23, 25, and 25. Under urgent-first service they are B, D, and F, with waits 25, 28, and 25. Both totals are three, but the affected visits differ.

Specify an output containing the rule name, cutoff 20, comparison “greater than,” count, and contributing visit IDs. Derive it from the already saved detail rows rather than simulating again.

Test the boundary explicitly: C's first-come wait is exactly 20 and must not be counted. An inclusive cutoff of at least 20 would count C as well and would answer a different question. The named lists make this a stronger check than comparing the equal totals of three.

Solution P12: Try One Change

[Hint P12 \(page 26\)](#)

Task. Set all six supplied priority labels to routine in a fresh copy. The two rules should now produce the same order and the same overall mean of 18 minutes. Both urgent-group means should be empty. This checks that the difference between the rules really comes from the supplied labels.

Solution. Set every priority to routine. Both rules must now choose A, B, C, D, E, F and yield the first-come waits 0, 15, 20, 23, 25, and 25. The overall and routine-group means are 18 minutes, the maximum is 25, and both urgent-group means are unavailable.

Compare the detailed order as well as the summary. Matching means alone could conceal a different ordering with the same aggregate. This task tests whether the original rule difference really came from the supplied labels rather than a hidden hard-coded preference for visit IDs. The automated check verifies both sequences and the mean/empty-subgroup behavior.

Appointment Letters

Solution 13.1: Letters Retain Appointment Identity

[Hint 13.1 \(page 27\)](#)

Task. Match each of the three generated letters to its appointment ID and explain why L04 is held.

Solution. L01 generates the letter for Sample Person One, October 1 at 09:00, Teaching Clinic A, contact 555-0101. L02 generates the letter for Sample Person Two, October 1 at 10:30, Teaching Clinic B, contact 555-0102. L03 generates the letter for Sample Person Three, October 2 at 14:00, Teaching Clinic A, contact 555-0101.

L04 is held because its contact field is missing. It must remain in the build report with that reason, rather than disappearing or borrowing another appointment's contact. The result is three generated letters and one held request from four input rows.

Compare each letter with its own appointment ID and all supplied fields. A familiar-looking name or correct clinic address is not enough to establish that the letter belongs to the right request. The generator creates documents only; this exercise does not send them.

Solution 13.2: The Same Name Does Not Mean the Same Appointment

[Hint 13.2 \(page 27\)](#)

Task. Give two appointments the same invented display name but different IDs and times. Predict the number of letters and check their separate identities.

Solution. For a concrete test, change L01 and L02 to the same invented display name while keeping their distinct IDs, times, locations, and contacts. Both letters must still be generated separately. With the rest of the baseline unchanged, there are three generated letters in total and L04 is still held.

L01 must retain 09:00, Clinic A, and 555-0101; L02 must retain 10:30, Clinic B, and 555-0102. Their equal display names must not cause one file or page to overwrite the other or cause their fields to be merged.

Check the build report's appointment identities and inspect both documents. A safe naming or indexing scheme incorporates the unique appointment ID. Do not

deduplicate requests by a human-readable name; the file has not supplied that name as a unique identifier.

Solution 13.3: Repair the Generator, Not One Letter

[Hint 13.3 \(page 27\)](#)

Task. L02’s letter uses the contact from L01. Specify a repair to the generator, not merely to the final page, and a check of every earlier letter.

Solution. Trace the contact placeholder to the row being rendered. A suitable repair ensures that each letter’s contact comes from that same appointment record, without a retained variable from the previous iteration or a lookup keyed by a nonunique display name. Regenerate every letter from the corrected generator.

Check L01 against 555-0101, L02 against 555-0102, and L03 against 555-0101, together with each letter’s ID, date, time, name, and location. L04 must remain held, not receive the last valid contact. Reverse the input order and repeat to expose any remaining dependence on a previous row.

Preserve the faulty output, the minimal L01/L02 case, the repair description, and the regenerated results. Editing L02’s final page alone may make that one page look right, but it leaves the bug available to corrupt every future run.

Solution 13.4: A Cover Sheet and a Print-Layout Contract

[Hint 13.4 \(page 27\)](#)

Task. Add a review cover sheet and test a long invented name and location in print preview. Define what happens when a letter would otherwise spill onto an additional page.

Solution. A workable cover sheet lists the synthetic batch identifier, creation date, generated appointment IDs, held IDs with reasons, and a prominent “Draft for review; not sent” label. It should precede the letters and not change which requests qualify for generation.

One explicit overflow policy is to allow a letter to continue onto a second page at the same readable text size. Repeat the appointment ID on each continuation page, number the pages, and start the next appointment on a new page. Do not clip content or shrink it below the agreed text size to force one page. A different one-page-only policy would need to hold overlong letters for manual layout review rather than silently omit text.

Test a long invented name and location, an ordinary letter, and the held L04 request in print preview. Check page breaks, margins, continuation identity, and the cover-sheet totals. These are visual acceptance checks; a successful Python run alone does not establish that the printed pages are usable.

Solution P13: Try One Change

[Hint P13 \(page 28\)](#)

Task. Supply the invented contact 555-0101 for L04. Four letters should now be generated and no request held. Check the added letter's details. The change must not add message-sending behavior.

Solution. Give L04 the invented contact 555-0101 and rerun in a fresh copy. All four requests now generate letters, and no request is held. The new letter must identify L04, Sample Person Four, October 2 at 15:00, Teaching Clinic A, and the supplied contact.

Check that the other three letters retain their own details. The program should still report that no messages were sent. Supplying a contact for document generation is not authorization to add sending behavior. The automated check verifies that all four build-report rows are generated; visual review remains necessary for the rendered letters.

Rule-and-Boundary Lab

Solution 14.1: A Total Can Hide Incorrect Components

[Hint 14.1 \(page 29\)](#)

Task. Reconstruct C03's three components rather than checking its total alone.

Solution. C03 has duration 10, task count 6, and support present no. Under the fictional rule, duration 10 receives two points; six tasks receive two points; and absent support receives one point. The total is therefore five.

Require the output to show components 2, 2, and 1, not only total 5. An incorrect pair of component values could accidentally sum to the right total, so checking the total alone would miss the implementation error.

The boundary convention matters: a duration of exactly 10 belongs in the two-point interval, and a task count of exactly 6 belongs in the two-point task group. These are supplied classroom rules with no clinical interpretation.

Solution 14.2: Task-Count Boundaries

[Hint 14.2 \(page 29\)](#)

Task. Keep duration at four and support at yes; test task counts 2, 3, 5, and 6. Predict all four totals.

Solution. Duration four contributes zero, and support yes contributes zero. For task counts 2, 3, 5, and 6, the task component is respectively 0, 1, 1, and 2. Those are also the four totals.

Use four distinct case IDs, keeping the other two fields identical. The cases establish the transitions after 2 and after 5: at most 2 gives zero, 3 through 5 gives one, and 6 or more gives two.

Check each component and total in the saved report. If task count 5 receives two, the upper boundary of the middle interval is wrong. If 6 still receives one, the next interval begins too late. Including both sides of each transition makes this more informative than testing widely separated values.

Solution 14.3: Missing Is Not Zero Duration

[Hint 14.3 \(page 29\)](#)

Task. A calculator gives C05 zero points. Explain why a missing duration is not a zero-duration observation and identify which output fields must be blank.

Solution. C05 has no supplied duration. A blank field supplies neither a zero-duration observation nor evidence for the lowest scoring category. The row must have review status with the missing-or-nonnumeric-duration reason.

Under the reference program's all-inputs-valid calculation rule, duration_points, task_points, support_points, and total are all blank. Even though task count and support are supplied, the program does not return a partly calculated score as though it were a completed result.

Compare C05 with a separate valid case having duration 0, task count 3, and support yes. That case can be calculated and has total 1. This contrast shows why automatically converting an empty string to zero is a substantive change to the meaning of the input, not a harmless convenience.

Solution 14.4: A Changed Cutoff Needs Changed Expectations

[Hint 14.4 \(page 29\)](#)

Task. Change the upper duration cutoff to 12. Revise the expected cases for 10, 11.9, and 12 and explain why retaining every old expectation would be wrong.

Solution. Keep the lower duration cutoff at 5 and change the upper cutoff from 10 to 12. The new duration points are zero below 5, one from 5 through values strictly below 12, and two from 12 upward.

For durations 10, 11.9, and 12 the duration components are therefore 1, 1, and 2. With task count 2 and support yes, these are also the totals. For the original C03, whose other components total three, the new total becomes four instead of five.

Retain old expectations for unchanged input validation and task/support rules, but revise those whose meaning depends on the old duration cutoff. Save the old specification and tests as the old version. Editing tests merely to make an unchanged program pass would be wrong; here both a declared rule change and its implementation justify the new expectations.

Solution P14: Try One Change

[Hint P14 \(page 30\)](#)

Task. Change C02's duration from 5 to 4.9. Its duration component and total should fall from 1 to 0; every other case should remain unchanged. This is a test of the stated interval boundary, not a new scoring recommendation.

Solution. Change only C02's duration from 5 to 4.9. Its duration points decrease from one to zero. Its task and support components were already zero, so its total also decreases from one to zero.

C01, C03, C04, C05, and C06 should have exactly the same derived fields as before. In particular, the invalid and missing-duration cases remain review rows rather than becoming zero scores. The automated check compares every unaffected case with the baseline and inspects C02's changed component and total.

Predictive-Value Explorer

Solution 15.1: Reconstructing the Four Expected Cells

[Hint 15.1 \(page 31\)](#)

Task. At 10 percent prevalence, reconstruct all four expected counts and both condition-group totals.

Solution. The model has 10,000 people, prevalence 10 percent, sensitivity 90 percent, and specificity 95 percent. There are therefore 1,000 expected people with the condition and 9,000 without it.

Among the 1,000, true positives are 900 and false negatives are 100. Among the 9,000, false positives are 450 and true negatives are 8,550. Check both group totals: $900 + 100 = 1,000$ and $450 + 8,550 = 9,000$. All four cells sum to 10,000.

The expected positive total is 1,350, so the positive predictive value is $900/1,350$, approximately 66.7 percent. These are expected counts under supplied assumptions, not a measured cohort or a diagnosis for a person. Retain the population and assumptions beside the calculation so that the percentages cannot be mistaken for interchangeable quantities.

Solution 15.2: A Five-Percent Scenario

[Hint 15.2 \(page 31\)](#)

Task. Use 5 percent prevalence with the same model. Predict the true and false positive counts before comparing the browser and CSV calculation.

Solution. At prevalence 5 percent, 500 of the model's 10,000 people are expected to have the condition and 9,500 not to have it. At 90 percent sensitivity, true positives are 450 and false negatives 50. At 95 percent specificity, false positives are 475 and true negatives 9,025.

There are 925 expected positives, giving positive predictive value $450/925$, approximately 48.649 percent, displayed as 48.6 percent. Check $450 + 50 = 500$, $475 + 9,025 = 9,500$, and the four-cell sum 10,000 before comparing the browser and CSV.

A result with 25 false positives has applied the five-percent false-positive rate to the wrong group. A discrepancy caused only by rounding should be distinguished from a different population or an incorrect cell calculation.

Solution 15.3: Specificity Is Not the Predictive Value

[Hint 15.3 \(page 31\)](#)

Task. A student says that 95 percent specificity makes 495 false positives impossible. Explain the mistake using the 9,900 people without the condition.

Solution. In the one-percent-prevalence scenario, 9,900 modelled people do not have the condition. Specificity 95 percent means that 95 percent of that group are expected to test negative and 5 percent positive. Five percent of 9,900 is 495, so 495 false positives is exactly what these assumptions predict.

The 100 people with the condition contribute 90 true positives at 90 percent sensitivity. Thus expected positives total 585 and the positive predictive value is $90/585$, about 15.4 percent. The apparently surprising result comes from applying rates to groups of very different sizes.

The student's mistake is treating specificity as the fraction of positive results that are true positives, or as a guarantee of only a tiny absolute false-positive count. The two percentages condition on different groups. The model illustrates that distinction without establishing any real test's performance.

Solution 15.4: Editable Assumptions and an Undefined Result

[Hint 15.4 \(page 31\)](#)

Task. Propose editable test characteristics. Define what the page must show when there are no expected positive results, and test that invalid input does not leave an old result on screen.

Solution. A suitable extension accepts prevalence, sensitivity, and specificity as finite percentages from 0 through 100, with a clearly stated positive model-population size. Recompute all four expected cells from those inputs. Reject blanks, nonnumeric text, nonfinite values, and out-of-range percentages with a specific message.

When true positives plus false positives is zero, show “Positive predictive value unavailable: no expected positive results.” Leave the numerical predictive-value field blank; do not display zero percent. For example, sensitivity 0 and specificity 100 produce no expected positives at any permitted prevalence. The four expected cells and their total can still be displayed.

Start with a valid scenario, then enter an invalid percentage such as 101 or blank text. The old cells and predictive value must be cleared or explicitly marked unavailable before the error is shown. Restore a valid value and require a fresh calculation. This stale-result test is an interactive browser check, separate from testing the Python arithmetic.

Solution P15: Try One Change

[Hint P15 \(page 32\)](#)

Task. Add scenario rows with prevalence 0 and 100. Under the unchanged baseline assumptions, the corresponding saved positive predictive values are 0% and 100%. Check all four expected cells and their total as well. The companion's automated checks verify the Python results; use the browser checklist to check the interactive controls yourself.

Solution. Add prevalence rows 0 and 100 to the scenario CSV. At 0 percent the expected cells are true positive 0, false positive 500, false negative 0, and true negative 9,500; the positive predictive value is 0 percent. At 100 percent they are 9,000, 0, 1,000, and 0 respectively, giving 100 percent.

Both scenarios still have expected positives, so neither is the no-positive-results case in Exercise 15.4. Check the four-cell sums and the relevant group totals, not only the displayed predictive values. The automated check verifies all five numbers at both endpoints in the Python output. Interactive controls should be checked separately in the browser.

Literature-Review Organizer

Solution 16.1: Retrieved Records and Study Groups

[Hint 16.1 \(page 33\)](#)

Task. Reconcile the eight retrieved records with the six study groups without deleting either repeated-study pair.

Solution. ST01 contains R01 and R02. ST02 contains R03. ST03 contains R04 and R06. ST04 contains R05. ST05 contains R07. ST06 contains R08. The group sizes are 2, 1, 2, 1, 1, and 1, summing to eight retrieved records across six study groups.

The baseline decisions are include for ST01, ST03, and ST06; exclude for ST02 and ST05; and pending for ST04. There are three included groups, two excluded, one pending, and no conflicting group.

The additional records in ST01 and ST03 are retained in their record-ID lists and source locators. Grouping is not deletion. Check that the union of all grouped record IDs contains each of R01–R08 exactly once and that no supplied source information has been silently discarded.

Solution 16.2: A Conflicting Year

[Hint 16.2 \(page 33\)](#)

Task. Change only R02’s year to 2025. Predict the decision counts and require the review output to name the field and both supplied values.

Solution. Change only R02’s year from 2024 to 2025. ST01 now contains different supplied years: R01 has 2024 and R02 has 2025. Its group decision becomes review conflict, and the conflict-fields column must name year. The supplied-differences field should preserve both values, with the record IDs and source locators still available.

There remain eight retrieved records and six groups. The decision counts become include 2, exclude 2, pending 1, and review conflict 1. ST01 alone moves from include to conflict; ST03 and ST06 remain included.

Do not resolve the disagreement by taking the first year, selecting the more recent year, or deleting R02. The exercise supplies no evidence for preferring either value. The Python Companion’s related task changes R02 to 2023 instead; it has the same decision counts but different conflicting values, so keep the two fixtures distinct.

Solution 16.3: A Study, a Report, and a Retrieved Record

Hint 16.3 (page 33)

Task. An organizer deletes a conference abstract because a later report has the same study ID. Explain what evidence can be lost and distinguish a study, report, and retrieved record.

Solution. A study is the underlying investigation. A report is a particular document or account of it, such as an abstract or a later full report. A retrieved record is an entry obtained during searching or importing, which may point to a report already represented by another retrieval.

A conference abstract can contain details, dates, analyses, or wording that differ from the later account. The shared study ID alone does not establish that it is redundant. Deleting it can remove the evidence needed to identify a discrepancy or explain a screening decision.

Retain retrieved-record IDs and source locators, group records under the study ID, and document relationships between reports. Decide how duplicates or superseded reports are handled only under an explicit review rule. The organizer can expose agreement and conflict; it should not invent a judgment that the later document makes every earlier record dispensable.

Solution 16.4: An Extraction Worksheet with Stable Sources

Hint 16.4 (page 33)

Task. Design an extraction worksheet with an explicit row unit and stable source locators. State how pending and conflicting decisions affect entry into that worksheet.

Solution. One suitable design uses one row per study–report–outcome–timepoint combination. Include study ID, report ID, retrieved-record ID, a stable document identifier or stored-file identity, and a precise source locator such as page/table/section. Add the extracted field, supplied value and unit, extractor, extraction date, and review status. Keep verbatim source text where needed to support an interpretation.

Under one explicit policy, only groups with a resolved include decision enter the active extraction worksheet. Pending and conflicting groups remain in the screening register with the unresolved reason and an action required; they are not silently converted to exclusions or admitted as settled evidence. A provisional worksheet can be used instead, but its status must be unmistakable.

Test a study with two reports and two timepoints. The worksheet must retain the distinctions and lead each value back to its exact source. A study-level average without those links is not a substitute for an extraction record.

Solution P16: Try One Change

[Hint P16](#) (page 34)

Task. Change only R02's year from 2024 to 2023. ST01 should become review conflict with year named as the conflicting field; both R01 and R02 must remain. Expect two included groups, two excluded, one pending, and one conflict. Do not resolve the conflict by deleting R02.

Solution. Change R02 to 2023. ST01 should have review-conflict status with year named as the conflicting field and both supplied years, 2024 and 2023, retained. R01 and R02 remain together in the record-ID and source-locator fields.

The group totals are two included, two excluded, one pending, and one conflict, while all eight retrieved records remain represented. The automated check inspects the counts, both record IDs, the conflict field, and both years. Resolving the conflict by deleting the revised row would defeat the task even if it restored the original include count.

Reproducible Research Table

Solution 17.1: Direction, Pairing, and the Five Changes

[Hint 17.1 \(page 35\)](#)

Task. Check S01’s subtraction direction and reconstruct the five-change mean and median.

Solution. S01 has baseline 10 and follow-up 8, so its change is 8 minus 10, or -2 teaching units. The complete-pair changes are S01: -2, S02: -3, S03: -3, S05: -4, and S06: -2. They sum to -14, giving mean $-14/5 = -2.8$.

In increasing order they are -4, -3, -3, -2, -2, so the median is -3. S04 remains among the six people but does not contribute a complete pair because its follow-up value is missing.

Check each person’s two source locators and subtraction direction before checking the aggregate. The summary must distinguish six people from five complete pairs. A statement that the average “improved” would require a meaning for the measurement direction that this arithmetic exercise has not supplied; the correct description here is simply a mean change of -2.8 teaching units.

Solution 17.2: A Missing Record Is Not a Blank Record

[Hint 17.2 \(page 35\)](#)

Task. Remove S04’s follow-up record instead of leaving its value blank. Predict which derived source locator changes and which complete-pair counts do not.

Solution. Remove S04’s follow-up record, originally source data row 8, instead of leaving that row’s value blank. S04’s derived follow-up remains blank, its paired flag remains 0, and its change remains unavailable. Its `followup_row` locator changes from 8 to blank because there is no longer a source record for that timepoint.

The input record count falls from twelve to eleven. The people count remains six; complete pairs remain five; mean change remains -2.8 and median change -3. The source-row locators for later records also shift: S05’s baseline/follow-up become rows 8/9 and S06’s become 10/11.

This is why locators must identify both a source version and a row. A bare row number is not a permanent identity across edited files. Preserve both input versions

and their hashes, and compare stable person/timepoint identities when deciding which derived measurements are unchanged.

Solution 17.3: A One-Person Sign Check

[Hint 17.3 \(page 35\)](#)

Task. A report has mean change +2.8 but the same absolute magnitude as the reference. Name the smallest source-level check that detects the sign error.

Solution. Inspect S01 alone: baseline 10 and follow-up 8 require change -2 under the declared follow-up-minus-baseline rule. A derived change of +2 immediately exposes the reversed subtraction, without recomputing the entire table.

Then check the other complete pairs and regenerate the summary from the corrected derived changes. Their mean is -2.8, not +2.8. Equal absolute magnitudes do not make the two summaries equivalent; they describe opposite directions under the same definition.

Save this one-pair case as a regression test with its explicit input values and expected signed output. An assertion that checks only the absolute difference, or a summary that discards the sign in formatting, would fail to protect the intended rule.

Solution 17.4: Format Only at the Presentation Boundary

[Hint 17.4 \(page 35\)](#)

Task. Add a formatted summary with teaching units and one decimal place. Keep the unrounded CSV and specify how the formatted values will be checked against it.

Solution. One suitable display reports: people 6; complete pairs 5; mean change -2.8 teaching units; median change -3.0 teaching units. State that change is follow-up minus baseline and that only complete pairs contribute to those two summaries.

Keep the derived CSV and computational summary at their original numerical precision. Create the one-decimal display from those saved values rather than rounding each person's change first and then averaging. Include a basis count with each displayed summary where ambiguity is possible.

Check each formatted string against the agreed rounding rule applied to the unrounded source value. Also test a changed fixture producing more decimal places and an empty-complete-pair case, where the display must say unavailable instead of formatting a nonexistent value as 0.0. Formatting should alter presentation, not population, units, or subtraction direction.

Solution P17: Try One Change

[Hint P17 \(page 36\)](#)

Task. Supply follow-up value 10 for S04's existing blank follow-up record. Expect six complete pairs, with S04's change -1. The mean and median both become -2.5. The original input and its run record must remain available as a separate version.

Solution. Supply follow-up value 10 for S04's existing blank follow-up row. Its baseline is 11, so its change is -1. The complete changes are now -2, -3, -3, -1, -4, and -2, with sum -15 and mean -2.5.

Sorted, they are -4, -3, -3, -2, -2, -1. The middle pair is -3 and -2, giving median -2.5. The people count remains six and the complete-pair count becomes six. S04's follow-up source locator remains row 8 because this task edits an existing value rather than adding or deleting a record.

The automated check verifies both summary values and S04's -1 change. Keep the earlier input and run record so that the transition from five to six contributing pairs remains inspectable.

Reminder Simulation

Solution 18.1: The First Two Trial Differences

[Hint 18.1 \(page 37\)](#)

Task. Recalculate the first two trial differences from the displayed attendance counts.

Solution. In trial 1, the usual group has 56 attendances and the reminder group 68. Their difference is 12 out of 80, so the attendance-rate difference is 100 times $12/80 = 15$ percentage points.

In trial 2, the counts are 52 and 67. The difference is 15 out of 80, giving 18.75 percentage points. Both use reminder minus usual, matching the sign convention of the saved trial column.

These are two realized trials from the stated simulation algorithm and seed, not the theoretical mean difference or a percentage increase relative to the usual-group count. Dividing 12 by 56 in the first trial would calculate a different relative-change measure. Check the first two saved rows against these hand calculations before interpreting the histogram.

Solution 18.2: A Single-Plan Program Must Reject Two Plans

[Hint 18.2 \(page 37\)](#)

Task. Add a second plan row. State whether the declared single-plan program should reject it or run it, and test the actual response.

Solution. Append a second valid-looking plan row in a fresh copy of `simulation_plan.csv`. The declared program should reject the input with the exactly-one-plan-row error. It must not silently use the first row, run only the last row, or combine the two plans.

Check the actual nonzero exit and that no new trial, summary, frequency, or histogram files were created in the fresh folder. Preserving the error is part of the test. A readable CSV with two rows is structurally valid but violates this application's single-plan contract.

Running several scenarios is a reasonable extension, addressed in Exercise 18.4, but requires explicit scenario identities, separate assumptions, and output rules. It is not implied by adding a row to an input format whose current meaning is one plan.

Solution 18.3: Which Bin Contains Zero?

[Hint 18.3 \(page 37\)](#)

Task. A histogram places zero in the bin ending at zero even though bins include their left endpoint. Explain the boundary error and reconcile the corrected bin counts.

Solution. With five-percentage-point bins of the form [lower, upper), zero belongs to $[0, 5)$, not $[-5, 0)$. Values exactly -5 belong to $[-5, 0)$; values exactly 5 belong to $[5, 10)$. The reference calculation uses five times the floor of difference/5 as the lower endpoint.

Recompute the bin assignment for every saved trial, not a second simulation. If z trials have exactly zero difference, a defective table that put those trials in $[-5, 0)$ must subtract z from that bin and add z to $[0, 5)$. The total trial count remains 1,000.

The saved reference run contains 18 zero-difference trials. Its correct counts are 55 in $[-5, 0)$ and 131 in $[0, 5)$. The particular mistake described would instead give 73 and 113, with the same grand total.

Check every bin against its member trial IDs and require that each trial occurs once. Equal grand totals do not establish correct boundary assignment. The count of reminder-not-higher includes zero differences, whereas the negative bin $[-5, 0)$ does not; those two quantities should not be confused.

Solution 18.4: Two Declared Scenarios

[Hint 18.4 \(page 37\)](#)

Task. Design a two-scenario comparison with scenario IDs, separate saved plans, and an explicit independence assumption. State which checks should remain unchanged when only the seed changes.

Solution. One model compares scenario A with probabilities 0.70 and 0.80 and scenario B with probabilities 0.70 and 0.75, using 80 appointments per group and 1,000 trials in each. Give them explicit scenario IDs and separate saved one-row plans. These are supplied fictional assumptions, not estimated effects.

Declare that appointments are independent within and between groups and that the two scenario runs are intended as separate simulated experiments. Use separately initialized random generators with recorded seeds; do not accidentally share generator state between scenarios or imply a paired common-random-number design without specifying one. Record the program and environment as well as each plan.

When only a seed changes, the schema, number of trials, group sizes, probabilities, subtraction direction, permitted attendance counts, and bin rule must stay unchanged. The realized trial counts, mean, quantiles, and bin frequencies may change. Reconcile every frequency table to 1,000 trials, but do not demand the previous seed's exact summary.

Solution P18: Try One Change

[Hint P18 \(page 38\)](#)

Task. Set both supplied probabilities to 0 and keep the other plan fields unchanged. Every trial must have zero attendance in both groups, all differences and central bounds must be zero, and reminder-not-higher must count all 1,000 trials. This is a deterministic boundary test of a random program.

Solution. Set both probabilities to zero while keeping the group size, repetition count, and seed unchanged. Each of the 1,000 trials must have usual attendance 0, reminder attendance 0, and difference 0. The mean and both central bounds are zero, and reminder-not-higher equals 1,000 because equality is included.

The frequency table has one occupied bin $[0, 5)$ with count 1,000. This also provides a particularly clear boundary check for Exercise 18.3. No realization variability remains in this fixture, even though the program still calls its random generator.

The automated check inspects every trial's three numerical outputs and all four summary values. A nonzero difference would expose an implementation or input-use error, not an unlucky random sample.

District Outreach

Solution 19.1: North: Observed and Reweighted

[Hint 19.1 \(page 39\)](#)

Task. Reconstruct North's crude and equally weighted percentages from its two age-band counts.

Solution. North has 450 reached among 900 eligible people under 65, giving 50 percent, and 90 among 100 aged 65-plus, giving 90 percent. Its observed totals are 540 reached among 1,000 eligible, so its crude percentage is 54 percent.

With equal reference weights, the standardized percentage is the average of the two age-specific percentages: one-half of 50 plus one-half of 90, giving 70 percent. This reweights the rates to an equally split reference population rather than North's observed 90/10 age composition.

Do not report 700 reached as an observed North count. The supplied observed numerator remains 540. Keep the crude percentage, both age-specific rates, reference weights, and standardized percentage separately labelled so that their different meanings are visible.

Solution 19.2: Changing the Reference Weights

[Hint 19.2 \(page 39\)](#)

Task. Use weights 0.7 and 0.3. Predict the four standardized results and identify which observed columns must not change.

Solution. North's reweighted percentage is 0.7 times 50 plus 0.3 times 90, giving 62 percent. South's is 0.7 times 60 plus 0.3 times 80, giving 66 percent. East has the same two age-specific rates as South and also gives 66 percent. West gives 0.7 times 50 plus 0.3 times 70, or 56 percent.

The observed reached and eligible counts must not change. Nor should the crude percentages: North 54, South 78, East 72, and West 56 percent. Only the reference-weight fields and standardized percentages change under this experiment.

Use the same ordered association between weights and age bands in all four districts and confirm that the nonnegative weights sum to one. North and West help detect an accidental interchange of the weights because their upper-band rates differ from their lower-band rates.

Solution 19.3: An Equal Weighted Average Hides Differences

[Hint 19.3 \(page 39\)](#)

Task. North and South have equal standardized rates. Use their supplied age-specific rates to disprove the claim that the districts therefore have equal rates in each age band.

Solution. With equal weights, North and South both have standardized percentage 70. Yet North's under-65 rate is 50 percent and South's is 60 percent. North's 65-plus rate is 90 percent and South's is 80 percent. They differ by ten percentage points in opposite directions.

The equal weighted sums therefore do not imply equality in either component. They show only that the chosen linear combination agrees. The crude rates differ too, because the districts' observed age compositions are different.

A complete comparison should retain the two age-specific values and state the reference weights. Collapsing the display to a single standardized number is useful for a defined summary but cannot establish that the underlying patterns are the same.

Solution 19.4: A Caption That States the Reference Population

[Hint 19.4 \(page 39\)](#)

Task. Write a caption for a crude-versus-standardized chart that names the reference weights and explains why the standardized percentage is not an observed count.

Solution. One suitable caption is: "Outreach percentages in four fictional districts. Crude percentages use each district's supplied reached and eligible counts. Standardized percentages apply equal reference weights, 0.5 for under 65 and 0.5 for 65-plus, to the two age-specific percentages. They describe that common reference composition and are not observed reached counts."

If Exercise 19.2's weights are used instead, change the caption to 0.7 and 0.3 and ensure the chart was actually rebuilt from that version. Retain the observed counts in the accompanying table so the reader can reconstruct the crude percentages.

The caption should not claim that standardization removes every difference between districts or establishes an intervention effect. It describes precisely which supplied compositional difference has been reweighted.

Solution P19: Try One Change

[Hint P19 \(page 40\)](#)

Task. Double both reached and eligible counts in both North rows. Expect North totals of 1,080 reached among 2,000 eligible, but unchanged crude 54.0% and standardized 70.0%. Scaling the counts should not alter their ratios. Do not apply the multiplier to only a numerator.

Solution. Double both reached and eligible counts in North's two rows. They become 900/1,800 and 180/200. The age-specific rates remain 50 and 90 percent. The district totals become 1,080 reached among 2,000 eligible, so the crude percentage remains 54.0 and the equal-weight standardized percentage remains 70.0.

Every other district's source values and outputs should remain unchanged. Check the new totals as well as the unchanged ratios; an implementation that ignored the changed input could retain the old percentages for the wrong reason. The automated check verifies North's two aggregate counts and both percentages.

Journey Tests

Solution 20.1: What the Twelve Tests Establish

[Hint 20.1 \(page 41\)](#)

Task. Match each of the twelve test names with the behavior it is intended to establish.

Solution.

Test	Behavior checked
clean_counts	All eighteen clean source records are retained and placed.
problem_reasons	The six specified review rows have the exact reasons; E18 retains supplied P01 and owner P03.
duplicate_correction	Correcting source row 14 clears both duplicate classifications and leaves four review rows.
same_day	A follow-up on its visit date is permitted.
missing_visit	Removing E16 leaves seventeen retained records; E17 and E18 both remain for review.
multiple_visits	An added C06 visit leaves nineteen records and all four C06 identities for review.
bad_header	A malformed schema is rejected.
row_order	Reordering source rows preserves classifications when compared by the fixture’s identifying key.
escaped_HTML	Supplied markup-like details appear as text rather than interpreted HTML.
person_selection	The P03 report has the specified six records and retains E18’s supplied person.
no_overwrite	An existing destination is rejected without changing earlier files.
repeat_run	Two fresh runs of the same fixture produce identical output bytes in this reference implementation.

These tests provide evidence for their particular fixtures and assertions, not proof of correctness for every possible file. Several also call a shared conservation check, so a deletion defect can fail more than the test whose name first suggests it.

Solution 20.2: Disable a Check and Demand a Failure

[Hint 20.2 \(page 41\)](#)

Task. Disable the date-order check in an isolated copy and predict which named test must fail before running it.

Solution. In an isolated copy, disable the comparison that adds *event before visit*. Predict failure of `problem_reasons`: source row 3, E03, would lose that required reason and would no longer be classified as expected.

The duplicate-correction test also depends on retaining this independent problem after the ID correction; its expected four review rows would fall to three. This is a useful reminder that a test may exercise more than the behavior named in its title.

Run the suite, preserve its nonzero exit and failed CSV rows, then restore the original checker and run again in a fresh output state. Do not edit the expected reasons to agree with the disabled check. A suite that still reports every test as passing has not demonstrated sensitivity to this defect.

Solution 20.3: A Vacuous Passing Test

[Hint 20.3 \(page 41\)](#)

Task. A missing-visit test checks all returned C06 rows but not their identities or count. Explain how deleting the entire group can make this weak test pass.

Solution. After E16 is removed, the remaining C06 records should be E17 and E18, both for review. A weak test that merely asks whether every returned C06 row has review status can pass when the defective checker deletes both rows: there are no returned rows that violate the condition.

Require the full seventeen-record output, the expected set of source-row locators, and the C06 identity set {E17, E18} before checking their statuses and reasons. Also compare original source fields at each locator.

The reference suite includes these conservation and identity assertions. They turn “every returned row is acceptable” into the stronger claim actually needed: every required row was returned, its evidence was preserved, and its classification is correct. This pattern applies to any filtered result that could accidentally become empty.

Solution 20.4: Preserving a Deletion Failure and Its Repair

[Hint 20.4 \(page 41\)](#)

Task. Introduce that deletion defect in a separate copy. Add or inspect the completeness assertion, preserve the failed CSV row, restore the program, and rerun the suite.

Solution. One deliberate defect filters out C06 records carrying *no valid visit for encounter* just before the checker returns its report. In the missing-visit fixture that deletes E17 and E18. The corrected test must fail rather than accept an empty C06 group.

Preserve the faulty program version, the fixture without E16, the failed `journey_tests.csv` row for `missing_visit`, and the actual execution output. The shared conservation assertion should report that all input records must remain. The narrow repair is to remove the deletion filter so the checker returns the complete report, with review reasons attached instead of discarded rows.

Run the restored suite and require all twelve tests to pass. Preserve both the failure and the successful rerun, not just the final PASS table. The worked model supplied with this companion includes the actual deletion-failure report and restored report; these are distinct from the Python Companion's same-day-comparison mutation.

Solution 20.5: Large-Data Check

[Hint 20.5 \(page 42\)](#)

Task. In a copy of the large file, change E000014's event ID to E000002. Predict the all-person and P0001-only results. Explain how a filter-before-validation defect could escape a test that checks only twelve selected rows.

Solution. The complete report has 12,000 retained records, 11,998 placed and two for review: both occurrences of E000002. One belongs to P0001 and the other to P0002. A P0001-only report still examines 12,000 input rows and selects twelve, but must place only eleven and retain its E000002 for review. Filtering first hides the second occurrence and can wrongly place all twelve. The large suite's `duplicate_before_selection` check asserts classification and identity, not only the selected count. The unchanged original remains 12,000 placed with no review rows.

Solution P20: Try One Change

[Hint P20 \(page 42\)](#)

Task. In a disposable copy of `journey_check.py`, change the event-before-visit comparison from `days[i] < days[anchor]` to `days[i] <= days[anchor]`. Run the suite in a fresh folder. It must report failure, including `same_day`; the defective comparison now rejects permitted same-day events. Restore the original program rather than changing expected answers to force a pass.

Solution. In a disposable checker copy, replace the event-before-visit comparison by less-than-or-equal. The suite must fail, including `same_day`. The faulty comparison also treats the visit's own date as too early relative to itself, so additional failures are expected; do not restrict the check to a single failed row.

Preserve the failure CSV before restoring the original comparison. Then rerun the unchanged tests and require twelve passes. The automated check performs the mutation, requires the failed same-day test, saves `deliberate_failure.csv`, restores the original program, and checks the successful rerun.

A test failure is the expected evidence for this deliberately defective version. The exercise passes when the suite detects the fault and accepts the restored program, not when the faulty program is made to produce a green report by changing expectations.

Demonstration Package

Solution 21.1: Source Entries and Generated Members

[Hint 21.1 \(page 43\)](#)

Task. Explain why six source entries produce eight members in the baseline archive.

Solution. The baseline source list has six entries: the three journey programs, the clean and problem CSV fixtures, and `expected_summary.txt`. The builder includes those six files and generates `RUN.txt` and `manifest.json`. That gives eight archive members.

The source-entry count and archive-member count therefore describe different sets. The manifest records the explicitly supplied source members and their purposes and hashes; it is not evidence that arbitrary neighboring files were included. Inspect the ZIP member list rather than estimating its contents from the working folder.

Also read the generated run instructions. Correct member counting and matching hashes establish file inclusion and identity, not that the named command is correct or that the enclosed source can run in a fresh extraction.

Solution 21.2: Adding Two Real Source Files

[Hint 21.2 \(page 43\)](#)

Task. Add a release note and an actual test report to the source list. Predict both member counts and inspect the generated manifest.

Solution. Create a release note and obtain the actual twelve-row `journey_tests.csv` from the unmodified Chapter 20 suite. Add both filenames and their purposes to the explicit source list. The source-entry count becomes eight; the archive contains those eight plus the generated `RUN` and `manifest`, so the member count becomes ten.

Inspect the new manifest rows and recompute their hashes against the actual included files. The test report should contain twelve named PASS rows from the recorded run. State the tested program version in the release note, so a report for a different source version is not mistaken for evidence about this package.

Check the archive's exact member set as well as its count. A missing new file and an unrelated extra file could still total ten. The companion's worked package checks use this explicit-inventory principle rather than collecting every file in a working directory.

Solution 21.3: Matching Hashes Do Not Make Instructions Correct

[Hint 21.3 \(page 43\)](#)

Task. All hashes match, but `RUN.txt` names an obsolete program. Explain why file identity has not established a usable handover.

Solution. All hashes can match while `RUN.txt` tells the reader to execute a program that is absent or obsolete. The archive may be an exact copy of a flawed package. Hash verification does not interpret the command, check its arguments, or establish that required imports and inputs are present.

Extract the archive into a new directory that cannot borrow files from the original working folder. Follow the printed instructions exactly and require the expected outputs and named checks. Record the actual command, exit status, and result.

The repair is to correct the package builder's instructions or the source inventory, rebuild the release, and test the new extraction. Editing a local copy of `RUN` without rebuilding the archive leaves the distributed error unchanged. Preserve the earlier release identifier so the corrected handover can be distinguished from it.

Solution 21.4: A Worklist Package, Not a Retitled Journey Package

[Hint 21.4 \(page 43\)](#)

Task. Package the worklist rather than the journey tool. Rewrite the file inventory and run instructions, then establish success by a fresh extraction and actual run.

Solution. The minimal runnable worklist package needs `followup_worklist.py`, `worklist_encounters.csv`, and instructions naming that program. Include the reporting date 2026-09-17 and fourteen-day window explicitly in the command, together with a release note and the expected six-encounter status identities.

After fresh extraction, run `python3 followup_worklist.py --as-of 2026-09-17 --window-days 14`. Require E01 and E06 recorded, E02 past its window, E03 still open, and E04/E05 for review. Inspect the generated CSV, HTML, and run record. A second execution in the same output location should not be used to test a fresh-run promise.

A worked delivery should include the worklist program and input, a new `RUN` file, an explicit manifest, and named acceptance checks. Save the program and input from Chapter 23 of the *Python Companion*; specify and test the package as this solution describes. No prebuilt worklist ZIP is assumed. Its instructions must be specific to the worklist rather than inherited from the journey demonstration.

Solution 21.5: Large-Data Check

[Hint 21.5 \(page 44\)](#)

Task. Starting from the eight-member baseline, add `journey_large.csv` and an independent `expected_large_summary.txt`, but no other files. Predict the source and archive counts, and explain which run instructions need revision before this is a usable large-data delivery.

Solution. There are eight source entries and ten archive members: the original six sources plus the large CSV and its expected summary, followed by generated `manifest.json` and `RUN.txt`. The new summary states 12,000 input and selected records, 12,000 placed, and zero review rows, and names the actual large file. Preserve the small problem-file instructions and add a separate large run into a new output folder. The reference package maker's instruction template is small-example-specific and must be revised deliberately for this variant. Extract the new archive into a fresh folder, run both inputs, inspect the actual results, and compare all eight source hashes. A ten-member release-note variant has different contents despite sharing this total.

Solution P21: Try One Change

[Hint P21 \(page 44\)](#)

Task. Create an unrelated file named `decoy.txt` beside the program, but do not add it to `package_files.csv`. Build the ZIP in a fresh copy. It should still have eight members and no `decoy.txt`. This checks that the explicit inclusion rule controls the package.

Solution. Create `decoy.txt` beside `package_builder.py` but do not add it to `package_files.csv`. Run the builder in a fresh chapter copy. The archive should still contain eight members, with six explicit source files and two generated members. The decoy must be absent.

Inspect the exact ZIP member names. If the decoy is present, the implementation has probably switched from explicit inclusion to collecting a directory's contents. That change can leak irrelevant or unintended material even when the intended files are all present. The automated check requires the eight-member count and the decoy's absence.

Review-Status Check

Solution 22.1: Three Blocker Rows, Four Reasons

[Hint 22.1 \(page 45\)](#)

Task. Reconcile three blocker records with four reasons in the original register.

Solution. Specification and User guide are record complete. Program is a blocker because current version 4 differs from reviewed version 3. Tests is a blocker because its decision is pending rather than accepted and its reviewed version is missing. Data handling is a blocker because its accepted review expired on September 1, before the September 17 reporting date.

There are therefore three blocker records but four reason occurrences: one for Program, two for Tests, and one for Data handling. All five required artifacts remain represented.

Count blocker rows and reasons separately and attach every reason to its artifact. The result checks the completeness of the supplied review register under its rules. It does not create a review, certify the underlying work, or establish permission for real use.

Solution 22.2: Two Blank Versions Are Not an Agreement

[Hint 22.2 \(page 45\)](#)

Task. Blank both current and reviewed versions in Specification. Predict its classification under the nonblank-version rule.

Solution. Blank both version fields in the Specification row. It becomes a blocker with the missing-or-mismatched-version reason. Although the two strings are equal as blank strings, they do not satisfy the required nonblank-version condition.

The register still contains five records. Only User guide remains record complete, and four records now have blockers. There are five reason occurrences overall because the changed Specification adds one to the original four.

A test that checks only `current_version == reviewed_version` would incorrectly accept the row. Require both fields to contain the declared nonblank identifiers before comparing them. Preserve the blanks in the output's source-version fields rather than inventing a version to make the record complete.

Solution 22.3: Acceptance Has a Date Boundary

[Hint 22.3 \(page 45\)](#)

Task. An accepted review expired before the reporting date. Explain why the decision string alone does not make the register row complete.

Solution. The accepted Data handling review is valid only through its supplied valid-until date. At reporting date September 17, its September 1 expiry is earlier, so the row is a blocker with review-expired reason despite its accepted decision and matching versions.

The reference rule uses expiry strictly before the reporting date. A review with valid-until September 17 would not be expired on September 17 under that convention; it would be expired on September 18. State and test that boundary rather than relying on an ambiguous word such as current.

Keep the reporting date, artifact versions, and rule version with the classification. A status calculated for one date should not be presented as valid forever. Renewing a review requires a new supplied review record; the classifier must not extend the date on its own.

Solution 22.4: An Action List That Does Not Invent Decisions

[Hint 22.4 \(page 45\)](#)

Task. Prepare an action list for your synthetic project that retains the reporting date, classifier rule version, artifact versions, and unresolved questions without inventing reviewer decisions.

Solution. One model action list uses reporting date 2026-09-17 and classifier version review-register-v1. For Program, record current version 4 versus reviewed version 3 and ask whether a review of version 4 exists. For Tests, record current version 4, blank reviewed version, and pending decision; request a completed review or an explicit unresolved decision. For Data handling, record version 1 and expiry 2026-09-01; request the appropriate current review record.

Include artifact, current and reviewed versions, blocker reasons, requested action, responsible role if supplied, open question, and evidence locator when available. Leave unknown people, dates, and decisions blank or explicitly unresolved. Keep the two complete rows visible in the accompanying register.

The action list should preserve what is known and make the next question clear. It must not replace pending by accepted, infer a reviewer from a role title, or renew an expiry merely because the project needs a clean report.

Solution P22: Try One Change

[Hint P22 \(page 46\)](#)

Task. In the invented Program row, change `reviewed_version` from 3 to 4 so it matches `current_version`. Expect three complete records and two blocker records. Tests still has its two reasons; the expired Data handling review is unchanged. This input change does not itself establish that a real review occurred.

Solution. Change Program's reviewed version from 3 to 4 so that it matches its current version. Under the fictional register's other unchanged fields, Program becomes record complete. Specification and User guide were already complete, giving three complete records and two blocker records.

Tests retains its two reasons, and Data handling remains expired. Thus the remaining blocker records have three reason occurrences. The automated check verifies the status totals and preserves the two Tests reasons.

The change note must say that the exercise supplies the revised register entry. The act of editing this CSV does not establish that anyone actually reviewed a program.

Follow-Up Worklist

Solution 23.1: The Baseline Worklist by Identity

[Hint 23.1 \(page 47\)](#)

Task. Identify the four baseline status groups by encounter ID, not only by their counts.

Solution. At reporting date September 17 and window fourteen days, E01 and E06 have follow-up recorded. E02 has no follow-up after its window. E03's window is still open. E04 and E05 require review.

E01's supplied follow-up is September 5. E06's is September 16, exactly fourteen days after its September 2 index date. E02's window ends September 15, before the reporting date. E03's ends September 24. E04 has a follow-up before its index date; E05 lacks a contact route and is therefore a review case rather than an ordinary no-follow-up row.

The counts are 2, 1, 1, and 2, summing to six, but the identity lists are the stronger check. Retain the source fields, due dates, reporting date, reasons, and rule version beside each classification.

Solution 23.2: Late but Recorded; Future and Unusable

[Hint 23.2 \(page 47\)](#)

Task. Give E01 a follow-up on September 17, then September 18 in a separate copy. Explain the valid late record and the future-record review case.

Solution. In one copy, give E01 a follow-up date of September 17. Its index date is September 1, so the follow-up is day 16, later than the fourteen-day matching window. It is nevertheless a valid recorded follow-up known by the September 17 reporting date, so the worklist status remains follow-up recorded.

In a separate copy, set the follow-up date to September 18. That is later than the reporting date. E01 must then be classified review data with a future-follow-up reason, rather than treated as recorded by September 17 or as definitively absent.

These cases distinguish a timeliness match, a valid recorded event, and inconsistent dated information. Test both source dates separately and retain the unchanged reporting date in the output. Moving the reporting date at the same time would change the question.

Solution 23.3: Equal Counts, Different Positive Sets

Hint 23.3 (page 47)

Task. The capstone matcher and worklist each contain eleven positive cases. Use C103 and C114 to show why those counts do not establish that the reports use the same definition.

Solution. C103 has a valid follow-up on September 17, day 16 after its index date. It is recorded by the reporting date but is outside the stage-A fourteen-day matching window. C114 has a within-window follow-up, but its missing contact route makes its worklist status review data.

Thus C103 belongs to the worklist's recorded-follow-up group but not the matched set. C114 belongs to the matched set but not the recorded-follow-up status group. Each report has eleven positives because these two differences compensate numerically.

Compare the sets by encounter ID. The recorded set is obtained from the matched set by removing C114 and adding C103. This explains why the two eleven-case totals agree without representing the same definition. Review status can overlap a valid match; it is not automatically an exclusion from the matcher or the stage-A weekly denominator.

Solution 23.4: A Complete Stage-B Model

Hint 23.4 (page 47)

Task. Complete stage B from your clarified brief. Deliver the specification, field map, row ledger, dated outputs, tests, one failed case and repair, and a package that succeeds after fresh extraction.

Solution. A complete worked model appears in the Integrated Capstone section of this companion with its source programs and inputs in the *Python Companion*. It labels the teaching decisions explicitly: retain the fourteen-day inclusive matching window, report as of September 17, 2026, and use only encounters whose windows have strictly closed for each weekly denominator. Keep the supplied final/provisional week flags.

The model retains forty source records and twenty worklist encounters, supplies the field map and denominator ledger, and gives the new expected weekly identity sets before testing them. The final-week result becomes 3/10, or 30 percent; the provisional week has no closed-window denominator. The recorded/matched/review distinctions remain intact.

The package includes the inputs, executable programs, dated expected results, tests, and a preserved failure-and-repair record. Its RUN instructions were followed after fresh extraction. It is one solution to the clarified teaching brief, not the only acceptable choice. The separate twenty-one-day/closed-denominator example from the Python Companion is also worked through, including its empty denominator.

Solution 23.5: Large-Data Check

Hint 23.5 (page 48)

Task. Design a worklist conversion from the large journey file as of September 17, 2026. What happens when contact route and owner are left blank? What changes when office and Teaching Team are explicitly supplied as fictional administration? Contrast both results with fourteen-day matching without inventing weekly status flags.

Solution. Use each visit's encounter ID and date, and the corresponding recorded follow-up date, retaining event-level source links. Blank contact routes and owners make all 3,000 encounter rows *review data*, regardless of their valid dates. With *office* and *Teaching Team* explicitly supplied for this fictional trial, all 3,000 become *follow-up recorded*: every follow-up is on or after its visit and before September 17. Those added fields must be recorded as teaching configuration, not claimed as source facts. The fourteen-day matcher still has 1,684 matches and 1,316 encounters outside that window. C000003 is recorded but not a fourteen-day match. No final/provisional flags are present in the journey source, so a weekly extension needs separately supplied decisions and a denominator. Preserve all 12,000 original events in the conversion ledger.

Solution P23: Try One Change

Hint P23 (page 48)

Task. Run `python3 followup_worklist.py --as-of 2026-09-25` in a fresh copy. E03 should now have no follow-up after window, giving two recorded, two passed-window, zero open, and two review records. On September 24 itself, E03 is still open under the inclusive-end rule.

Solution. Run the worklist with reporting date 2026-09-25. E03's index date is September 10, so its fourteen-day window ends September 24. With no follow-up and no review issue, it now becomes no follow-up after window.

The six statuses are recorded for E01/E06, past-window for E02/E03, and review for E04/E05. There are no open-window rows. On September 24 itself E03 would still be open, because the reporting date must be strictly later than the due date for the window to have passed.

The automated check verifies the September 25 identity and status counts. Retain the reporting date in every generated row and in the run record, so the changed result is not mistaken for a change to the source follow-up facts.

Larger Practice Files

These four worked tasks use the larger fixtures in the *Larger Practice Files* section of the *Python Companion*. R2, R8, R9, and R23 are local labels for the associated chapters; they are not additional main-book exercise numbers. Use the larger fixtures, not the tiny chapter inputs.

Solution R2: Thirty-Six Journey Records

[Hint R2 \(page 49\)](#)

Task. Reconcile every record in the Chapter 2 fixture in *Larger Practice Files* in the *Python Companion*. Explain the duplicate identifier and the difference between the supplied person and display owner.

Solution. The checked report contains all 36 records from twelve encounters and six display owners. Thirty are placed; six remain for review. Their exact identities and reasons are:

Row	Event	Encounter	Reason
5	Q005	Q02	Duplicate event ID
8	Q008	Q03	Missing event date
20	Q020	Q07	Invalid event date: June 31
27	Q027	Q09	Event before visit
29	Q005	Q10	Duplicate event ID
36	Q036	Q12	Person differs from visit

The two Q005 occurrences are different source records. Correcting only one identifier in a new copy requires reconsidering both rows. Q036 preserves supplied person P01, but Q12's visit establishes display owner P06. It therefore remains visible under P06 with its original person field and mismatch reason. Rewriting P01 to P06 would conceal the defect rather than explain it.

Require one output locator for every source row 1–36, compare each original field, and then compare the six review rows above. A count of six alone would accept six entirely wrong classifications. The supplied larger fixture was executed and its complete locator and review sets checked.

Solution R8: Matching Forty Source Records

Hint R8 (page 49)

Task. Match sixteen encounters with 24 follow-up records. Account for early and late dates, equal-day ties, repeat people, and the unknown encounter reference.

Solution. The sixteen encounter rows remain in the match report. Fifteen have a selected follow-up; C012 has none. The selected identities are:

Encounter	Selected	Encounter	Selected
C001	F01	C009	F09
C002	F02	C010	F10
C003	F19	C011	F11
C004	F20	C012	None
C005	F21	C013	F13
C006	F06	C014	F14
C007	F07	C015	F15
C008	F08	C016	F16

For C002 the equal-day tie is settled by the smaller string ID, F02. C007 likewise selects F07 rather than its equal-day alternative F23. F22 is 21 days after C006’s index date and is ineligible. C011’s day-fourteen follow-up F11 is eligible; C012’s only candidate F12 is on day fifteen and is not. F24 has an unknown encounter reference and must remain in the separate unmatched-follow-up report.

C001 and C002 share P01. Matching by person could mix these distinct encounters and choose a follow-up belonging to the other encounter. Use the explicit encounter reference even when the person and dates happen to agree. The complete candidate table explains both selected and unselected follow-ups; the encounter summary alone does not account for all 24 follow-up records.

The verification run compared all sixteen selected identities with the supplied `EXPECTED.json` and required F24 to be the orphan. A useful further test permutes the source rows while retaining the same source values: the day/ID rule should select the same follow-ups.

Solution R9: Move the Missing Observations

Hint R9 (page 49)

Task. Reconcile the forty-person cohort. Then move the same pattern of missing waits to another subgroup without changing the observed multiset. Explain why equal observed medians do not establish equal experience for the whole cohort.

Solution. S101–S104 are under eighteen; S105–S108 are not enrolled. The included set is S109–S140: 32 people. In the original fixture S109–S132 have recorded follow-up

and an observed wait; S133–S140 have neither. Thus both counts happen to be 24, and in this original file they also refer to the same people. That agreement is a property of this fixture, not a general rule.

The observed multiset contains six copies each of 4, 8, 12, and 16. The twelfth ordered observation is 8 and the thirteenth is 12, so the median is $(8 + 12)/2 = 10$. Its basis is 24 observed waits, not all 32 eligible people.

For a controlled alternative, swap only the `wait_days` values of S109–S116 with those of S133–S140, in corresponding order. The former eight values become blanks; the latter eight receive the existing sequence 4, 8, 12, 16, 4, 8, 12, 16. Leave ages, enrolment, and `followup_recorded` unchanged. No missing wait has been estimated: this is a deliberately altered synthetic fixture with the same observed multiset.

The changed report still has forty candidates, 32 eligible people, 24 recorded follow-ups, 24 observed waits, and median ten. But the observed-wait set is now S117–S140, while the recorded-follow-up set remains S109–S132. The sets intersect in sixteen people; eight belong only to each set. Equal counts do not establish identical membership.

For another view, separate eligible ages 29–44 from ages 45–60. Each subgroup contains sixteen people. Originally all sixteen younger people have a wait and only eight older people do. After the swap, eight younger and all sixteen older people have a wait. Both subgroups still have observed median ten in either fixture. Nothing here determines the unobserved waits or shows that their missingness is harmless. The original worked example recorded this comparison. To repeat it, save this fixture from the *Larger Practice Files* section of the *Python Companion*, make the stated swap in a copy, and retain the actual changed input and outputs.

Solution R23: Thirty-Six Worklist Encounters

[Hint R23 \(page 50\)](#)

Task. Reconcile the larger worklist as of September 17, 2026, with a fourteen-day inclusive window. Check exact status identities and all review reasons.

Solution. All 36 source encounters are retained, with nine in each status. The recorded group is W001, W005, W009, W013, W017, W021, W025, W029, W033. The passed-window group is W002, W006, W010, W014, W018, W022, W026, W030, W034. The still-open group is W003, W007, W011, W015, W019, W023, W027, W031, W035. The remaining every-fourth records are the review group:

Encounter	Review reason or reasons
W004	Missing contact route
W008	Follow-up after the reporting date
W012	Invalid follow-up date: September 31
W016	Follow-up before the index date
W020	Missing owner
W024	Invalid index date or window end
W028	Future index date
W032	Missing contact route; follow-up before index date
W036	Missing contact route; missing owner

These nine review records have eleven distinct row/reason occurrences. Count W032 and W036 once each as records, not twice because each has two reasons. The reference output describes W008 and W016 with the general reason *inconsistent follow-up date*; the supplied dates explain which inconsistency occurred.

W001's valid follow-up is later than its fourteen-day window but no later than the reporting date. Its status is therefore recorded, not passed-window-without-follow-up. The within-window matcher and the administrative worklist answer different questions.

The automatic check compared every group with **EXPECTED.json**. For the browser acceptance check, combine owner and status selections, compare the shown encounter IDs with the CSV, then choose a combination having no rows. Require an explicit empty message and zero count. This interactive check is not established merely by inspecting the HTML text.

The Integrated Capstone

[The capstone hints \(page 51\)](#)

This worked project supports main-book Exercise 23.4 and the integrated application in the Python Companion. It joins the matcher, weekly monitor, and worklist without treating their different classifications as synonyms. Stage A reproduces the supplied teaching contract. Stage B changes one reporting decision and supplies new expectations. A third case answers the Python Companion’s particular twenty-one-day example.

Stage A: Reproduce the Original Contract

[The Stage-A hint \(page 51\)](#)

The reporting date is September 17, 2026. Follow-up matching uses elapsed days zero through fourteen, inclusive. Each encounter belongs to the Monday week containing its index date. All encounters enter that week’s denominator. The week beginning August 31 is final; the week beginning September 7 is provisional, as explicitly supplied in the separate flags file.

There are twenty encounter records and twenty follow-up records. The source ledger retains all forty, identified by source filename and data-row number. The two weekly flags are counted separately. Repeated people do not merge their encounters, and repeated follow-up IDs do not collapse two source rows into one.

Eleven encounters have within-window matches: C101, C102, C107, C111, C112, C113, C114, C115, C116, C117, and C118. The worklist also has eleven recorded encounters, but its set replaces C114 by C103. C103’s valid September 17 follow-up is day sixteen: recorded, but not a fourteen-day match. C114 has a within-window match but lacks contact information and therefore requires review. Equal totals of eleven hide these two different sets.

The review set is C104, C105, C106, C108, C109, C114, C120. C104 has a follow-up before its index date; C105 has a future follow-up; C106 has an impossible date; C108 and C109 have the two occurrences of duplicated F09; C114 lacks contact; C120 lacks an owner. C110 has no follow-up after its window; C119’s window is still open through the reporting date. The worklist accounts for every encounter: $11 + 7 + 1 + 1 = 20$.

F11 points to unknown encounter C999. It remains in the source ledger with its reason rather than becoming a new invented encounter. F09’s two source occurrences also remain individually visible, but are withheld from matching under the stated duplicate rule.

Week beginning	Confirmed	Eligible	Review	Flag
August 31	8	16	6	Final
September 7	3	4	1	Provisional

The final-week numerator identities are C101, C102, C107, C111, C112, C113, C114, C115. Thus the pooled final result is $8/16 = 50\%$. The provisional $3/4$ is retained separately, not pooled into the final value. Review is an overlapping flag, not a quantity to subtract automatically from either numerator or denominator.

Stage B: A Closed-Window Model

The Stage-B hint (page 51)

Here is one explicit new brief: keep the reporting date, the fourteen-day matching window, and the supplied week flags, but include an encounter in a weekly denominator only when its window has strictly closed. The definition is

$$\text{reporting date} > \text{index date} + 14 \text{ days.}$$

The end date itself remains part of the open window. This model asks for a completed observation window before a weekly rate is formed; it is a supplied teaching decision, not a recommendation for any service.

Only C101–C110 have closed windows on September 17. Their index date is September 1 and their window ends September 15. The September 3 encounters end September 17 and are not yet closed. Later encounters are likewise outside the denominator. The denominator ledger must nevertheless retain all twenty encounters with an explicit inclusion flag and reason.

The final numerator becomes C101, C102, and C107, giving $3/10 = 30\%$. The included review set is C104, C105, C106, C108, C109, so that final week has five review encounters. The provisional week has zero confirmed and zero eligible encounters under the changed denominator rule; its percentage is unavailable.

The forty-row source ledger, twenty worklist encounters, eleven matches, and worklist statuses are unchanged. C114 still has a match and still requires review, but no longer enters this final-week denominator because its window remains open. Do not alter the matching code to make its totals equal a weekly numerator.

Create the working files from the *Integrated Capstone* section of the *Python Companion*. Record the changed choice in PROJECT_CARD.txt. The supplied adapter supports this contract:

```
python3 capstone_adapter.py --output stage_b_run --as-of 2026-09-17 --window-
↪ days 14 --denominator closed
```

The source ledger remains complete and the denominator ledger records inclusion. Compare the saved results with the Stage-B values given here. Before automating those checks, create a separately named test suite with these independently justified expectations; the printed `capstone_tests.py` is the Stage-A suite and must not be presented as a Stage-B test without revision. The output directory and report must not already exist. The package was independently extracted, these commands were executed, and all twelve named checks passed. The model also retains the original stage-A suite so readers can reproduce that separate contract.

The Twenty-One-Day Example

[The twenty-one-day hint \(page 52\)](#)

The Python Companion gives another possible command:

```
python3 capstone_adapter.py --output changed_run --window-days 21 --  
    ↪ denominator closed
```

With its default reporting date of September 17, no encounter has a strictly closed twenty-one-day window: even the earliest, beginning September 1, ends September 22. Both dated weeks therefore have zero eligible denominator records, and the pooled final percentage is blank or explicitly unavailable. Reporting zero percent would wrongly suggest an observed denominator with no successes.

There are now twelve within-window matches because C103's day-sixteen follow-up becomes eligible. Its worklist status was already recorded and stays recorded. The worklist has eleven recorded encounters, seven for review, two still open (C110 and C119), and none past the window. All forty source records and twenty encounter records remain. This alternative was executed and checked independently of the fourteen-day test expectations.

Preserve a Failure and Its Repair

[The failure-and-repair hint \(page 52\)](#)

For a deliberate failure experiment, change an isolated adapter copy. It admits a date to the worklist's recorded-date collection only when $0 \leq \text{elapsed} \leq \text{window}$. That incorrectly applies the match-window rule to recorded follow-up. C103's day-sixteen date disappears from its worklist input and its status wrongly becomes no-follow-up-after-window.

Stage-B checks of late follow-up and status identities must fail on that defect. The weekly 3/10 and the forty-row source ledger can still look correct: neither alone detects the wrong worklist status. This is why the tests compare named encounter identities and meanings, not only grand totals.

Restore the original nonnegative-date condition for recorded follow-up, leaving the matcher's fourteen-day condition intact. Execute the restored adapter into a new destination and rerun the independently specified Stage-B checks. Preserve your actual failure report, faulty worklist, source snapshot, execution record, and restored result in a clearly labelled folder. A faulty source snapshot may use a `.txt` suffix so it cannot shadow the correct imported program. Do not claim the experiment was performed until those outputs exist.

What the Handover Establishes

[The handover hint](#) (page 52)

A completed handover should contain the three input CSV files, the unchanged chapter programs, the adapter, separately named and actually executed test suites for the chosen contracts, the field map, explicit instructions, the chosen project card, actual reference outputs, and a manifest of file hashes. The program versions in the model remain identifiable; a deliberately changed rule is not disguised as the original release.

A successful automated run establishes the saved numerical and identity checks for these fixtures. It does not establish browser accessibility, correct print layout in every browser, service suitability, privacy approval, clinical safety, or authority to use real records. The book's interface acceptance checks still need a real browser and the stated test sequence. This handover is a completed teaching model, not an operational authorization.

Coverage and Checks

Exercise Coverage

Each part contains 98 main-book exercises and 23 chapter-level Python Companion tasks, in chapter order. Four independent practice tasks and the integrated capstone also appear in both parts. Every numbered main question matches this edition of the main-book source. Each hint has a linked solution, and each solution links back to its hint.

Exercises 3.5 and 3.6 address full-input versus selected counts and a controlled missing date. Exercise 8.5 reconciles the large conversion and two matching windows. Exercise 20.5 exposes filtering before duplicate validation. Exercise 21.5 specifies a large-data package; Exercise 23.5 distinguishes missing administration from recorded follow-up. The small exercises retain their record-level checks; Exercise 1.3 no longer assumes a lost event.

Execution Checks for This Edition

All 23 chapter baselines were executed. The small journey suite passed twelve checks, the integrated capstone suite twelve, and the large journey suite ten. The four independent practice fixtures were executed and checked against their stated reference records and summaries.

The large journey retained 12,000 placed events and no review rows; selection of P0001 retained twelve after whole-input validation. Large matching returned 1,684 matches at fourteen days and 3,000 at twenty-one days. The unchanged worklist classifier reproduced both explicitly specified administrative variants in Solution 23.5. A freshly extracted baseline journey package ran without borrowing files.

Separate Chromium checks exercised the supplied selectors, empty result, predictive-value inputs and validation, the 80-percent-sensitivity example, and three-page letter printing. The saved HTML contents were loaded through automation because this container blocks direct file-URL navigation. A reader's local open/reopen experience, other browsers, and accessibility remain distinct checks. The Python Companion describes the separate PDF-to-program reconstruction trial and its one corrected wrap-space transcription error. Neither technical trial substitutes for an independent novice following the instructions.

Solution 21.5 specifies a package extension; it does not claim that the unchanged builder produces that extension. Solution 23.5 specifies a conversion and checked classifier consequences, not an unmentioned general-purpose adapter. Open-ended

P-task and interface variations retain their worked reasoning; this edition does not claim that every proposed extension was implemented and rerun. None of these execution checks establishes professional approval.

Where to Obtain the Working Files

The *Python Companion* contains the complete program listings and small input listings in its numbered chapters, followed by the capstone and independent practice fixtures. Save the named listings as actual plain-text files, preserving indentation and data. Its key-line excerpts are explanations, not replacements for the complete programs.

The actual `journey_large.csv` is a separate download from the same companion website. It is not a folder inside a PDF or a set of records to regenerate from a prompt. No separate `models` folder, checker archive, or historical production manifest is needed to use this volume. Keep each source and its expected results together, and run changes in new working folders.